

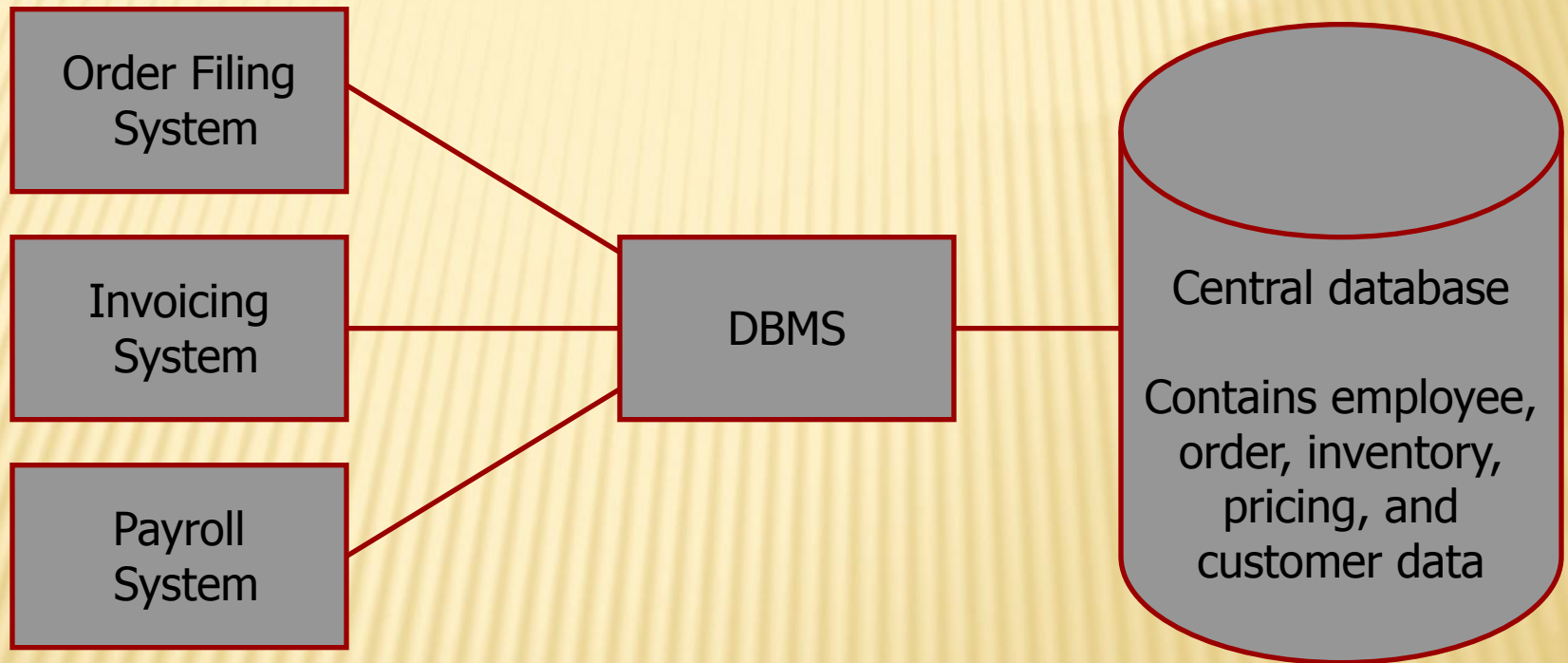
E-R DIAGRAM

Sara Tehsin



DATABASE MANAGEMENT SYSTEM

- A software system that is used to create, maintain, and provide controlled access to user databases



DBMS manages data resources like an operating system manages hardware resources



ELEMENTS OF THE DATABASE APPROACH (1)

- Data models
 - Graphical system capturing **nature and relationship** of data
 - 1. **Enterprise Data Model**—high-level **entities and relationships** for the organization
 - 2. **Project Data Model**—more **detailed view, matching data structure** in database or data warehouse
- Entities – **we met these concepts**
Week 1
 - Noun form describing a person, place, object, event, or concept

Understand entity **type** vs entity **instance**

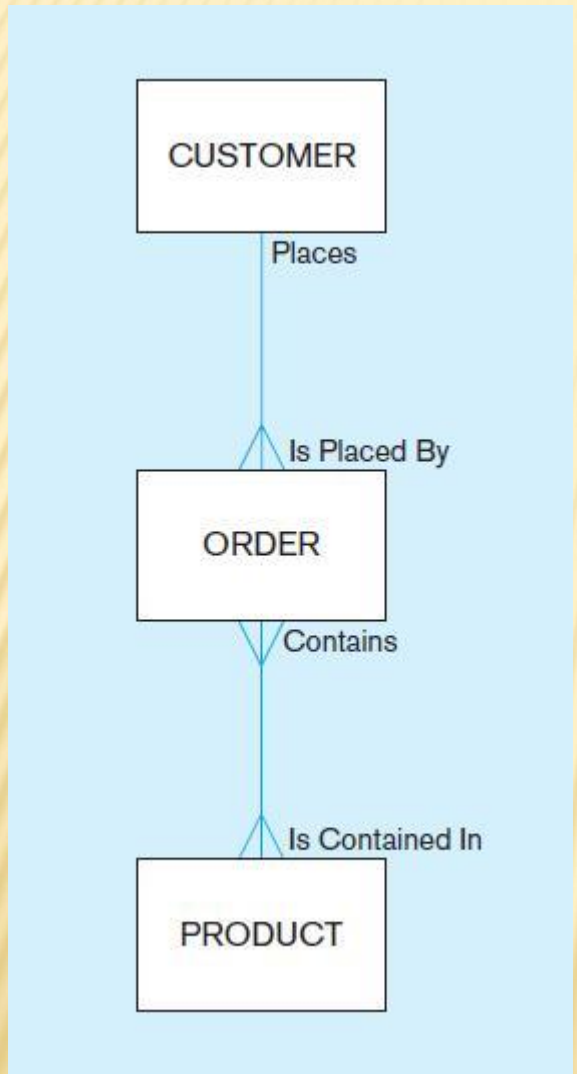


ELEMENTS OF THE DATABASE APPROACH (2)

- Relationships
 - Between entities
 - Usually one-to-many (1:M) or many-to-many (M:N)
- Relational Databases
 - Database technology involving
 - tables (relations) representing entities, and
 - primary/foreign keys representing relationships

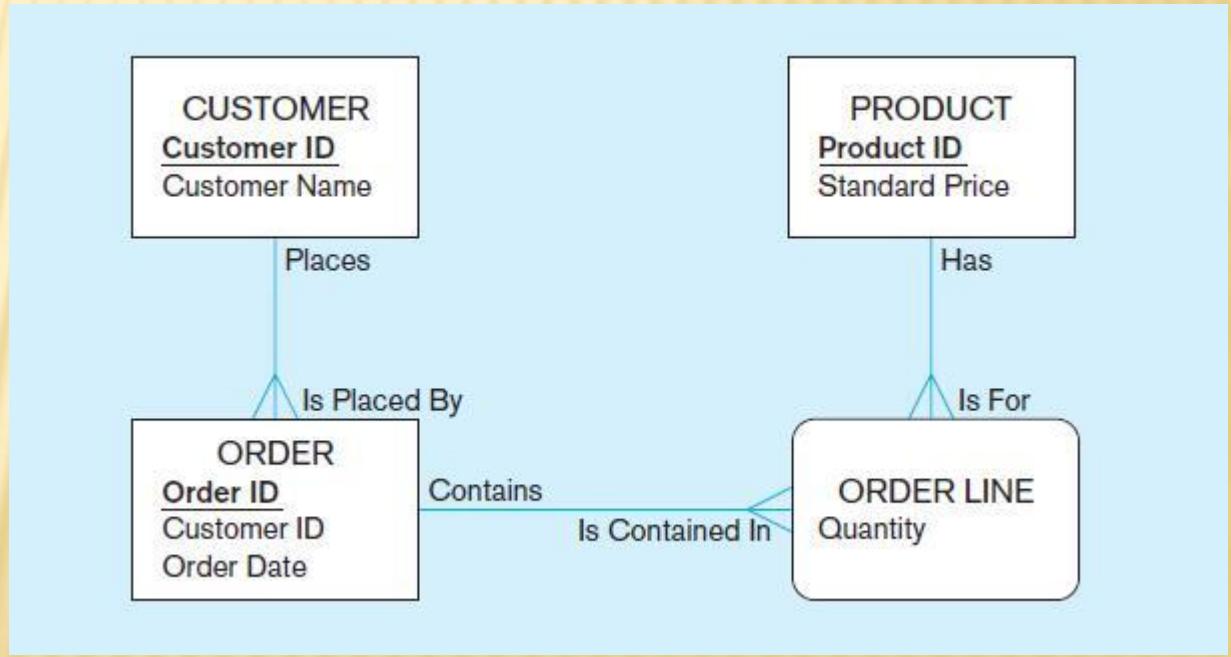


Fig 1-3 Comparison of enterprise and project level data models



Segment of an **enterprise** data model
high-level entities and relationships

Segment of a **project-level** data model
detailed view, matching data structure



ENTITY RELATIONSHIP DIAGRAM

- Logical representation of the data of an organization.
- **Entity:** An entity may be defined as a thing which is recognized as being capable of an independent existence and which can be uniquely identified.
 - Entities can be thought of as nouns.
 - Examples: a computer, an employee etc.
- **Relationship:** A relationship captures how two or more entities are related to one another.
 - Relationships can be thought of as verbs, linking two or more nouns.
 - Examples: an *owns* relationship between a company and a computer, a *supervises* relationship between an employee and a department etc.

ENTITY RELATIONSHIP DIAGRAM

- **Attributes:** Set of characteristics or properties of an entity which is of interest to the organization.
 - Student: Student_ID, Student_Name, Address etc.
 - Employee: Emp_ID, Emp_Name, Payroll etc.

ENTITY NAMES

- Entity names should relate to business, not technical characteristics.
- Entity names should be meaningful.
- The name used for the same entity should be same on all the ERDs on which the entity appears.
- Should be concise.

ENTITY NAME RULES

Identifiers are used to name various entities (entities or otherwise). Valid identifier names follow these rules:

- They have between 1 and 1024 characters long.
- They may contain letters, digits, underscores (_), spaces, dots (.), and dashes (-).
 - Identifiers consisting only of letters, digits, and underscores do not require quoting when the identifier is being referenced.
 - Identifiers containing at least one of (spaces, dots, or dashes).
- They are case-sensitive.

ENTITIES

- Strong Entity
 - Employee, Course
- Weak Entity
 - Dependents.
- Associative
 - Customer **order** Product

ER MODEL NOTATIONS

□ Strong Entity:

Entity Name

□ Weak Entity:

Entity Name

□ With Attributes

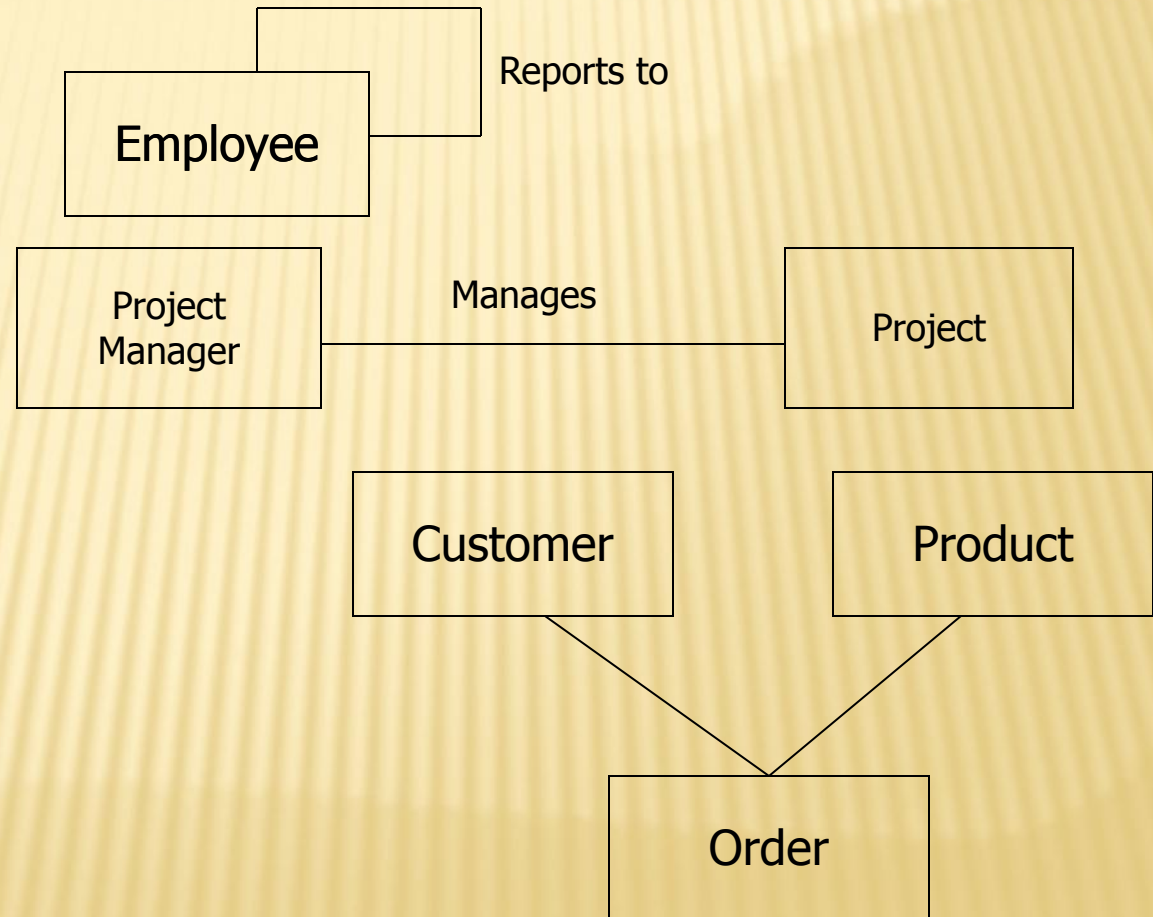
STUDENT
Std_ID
Std_Name
Std_Class
Std_RollNo.
Std_Address

DEGREE OF RELATIONSHIP

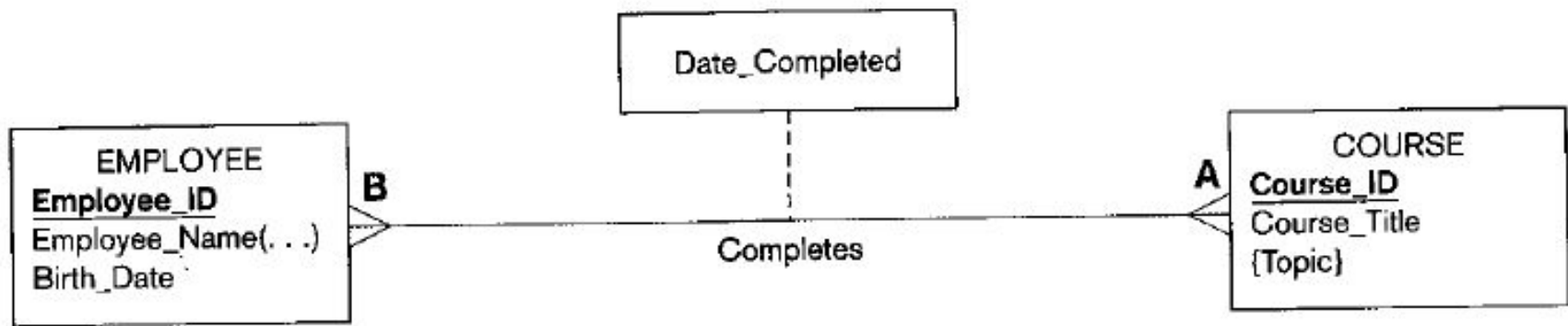
□ Unary

□ Binary

□ Ternary



DEGREE OF RELATIONSHIP



RELATIONSHIP CARDINALITY

Relationship cardinalities specifies how many of each entity type is allowed.

Mandatory One:

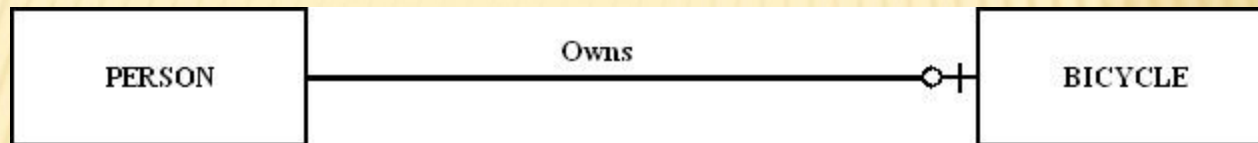
Mandatory Many:

Optional One:

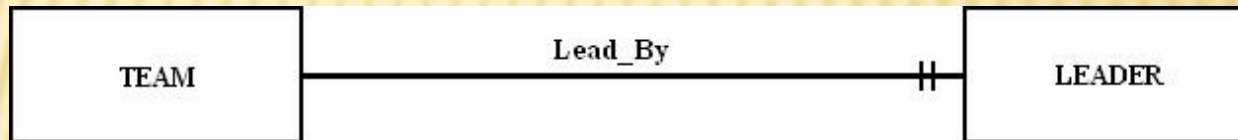
Optional Many:



RELATIONSHIP CARDINALITY

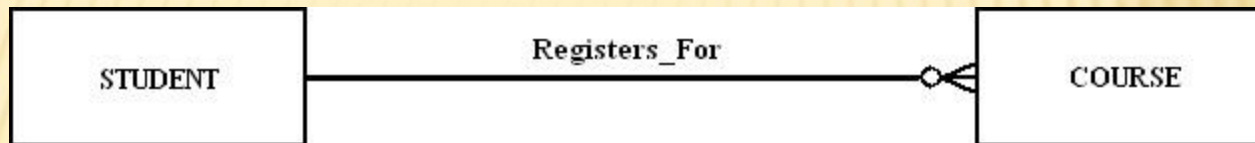


Optional One

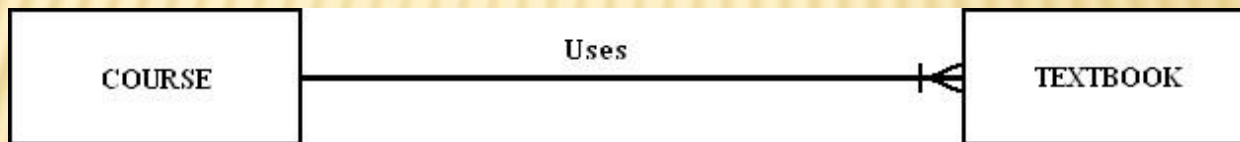


Mandatory One

RELATIONSHIP CARDINALITY



Optional Many

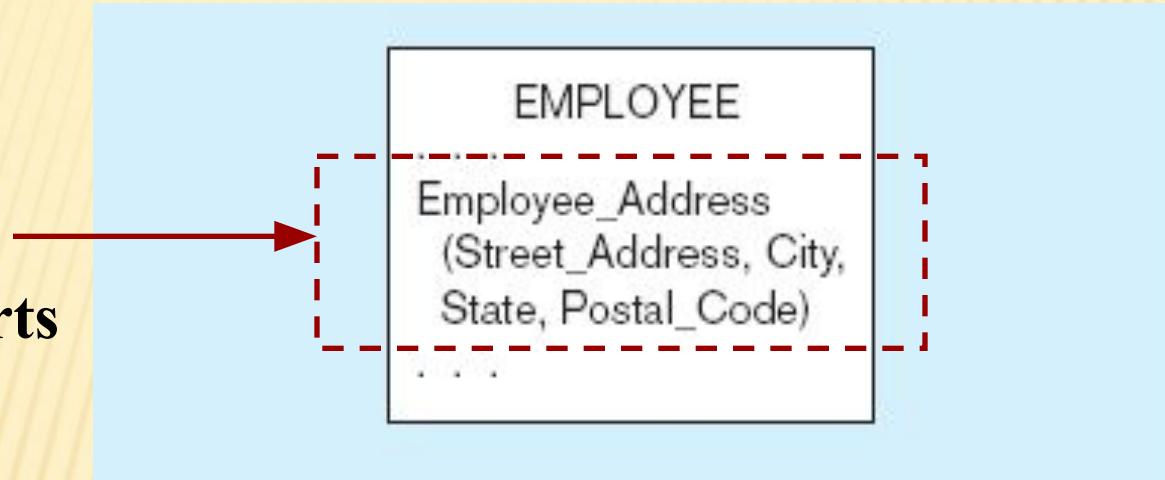


Mandatory Many

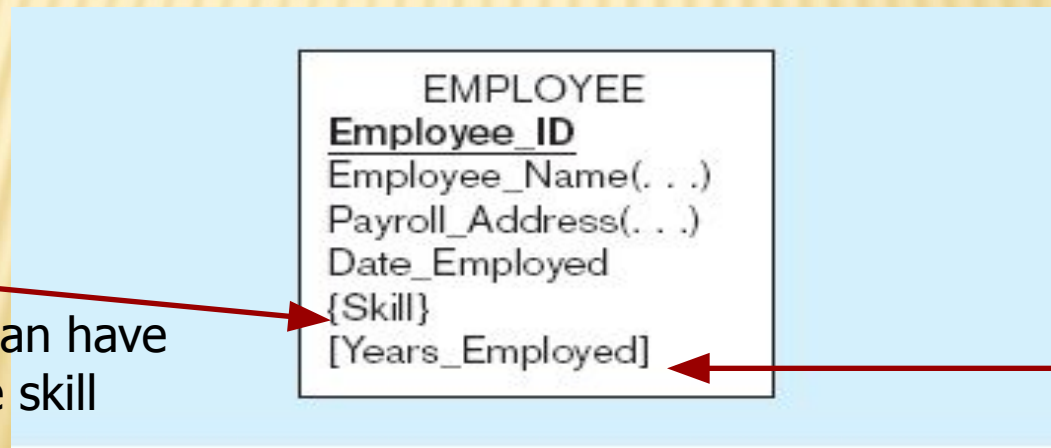
ATTRIBUTES

- **Composite Attribute:** that has meaningful component parts.
 - Example: Address (street No., House No., City etc.)
- **Multivalued Attribute:** that may take more than one value for the given entity.
 - Skill.
- **Derived Attribute:** whose value can be calculated from other attribute values.
 - $\text{Years_Employeed} = \text{Current_Date} - \text{Employeed_Date}$

**An attribute
broken into
component parts**

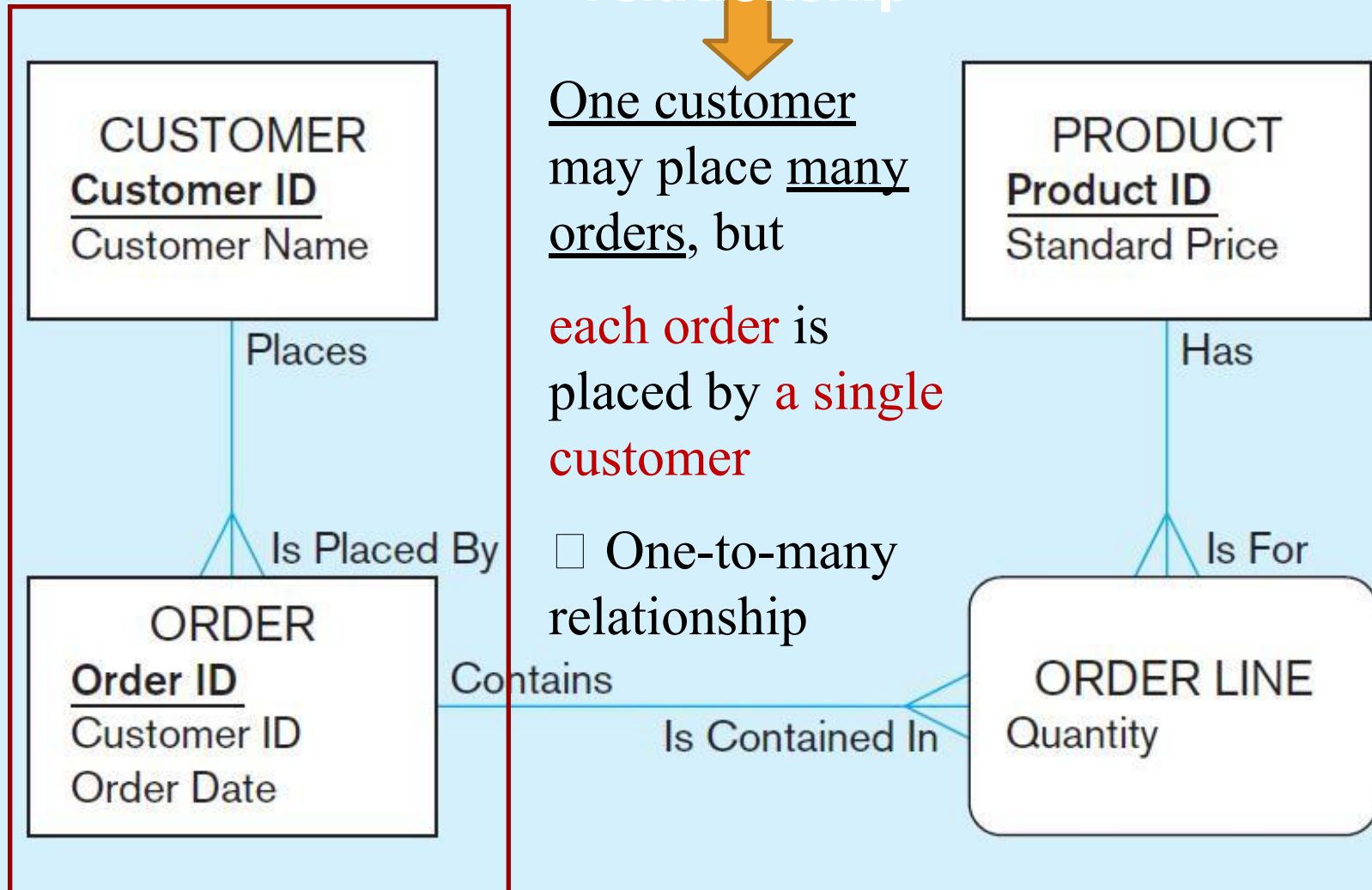


Multivalued
an employee can have
more than one skill



Derived
from date
employed and
current date

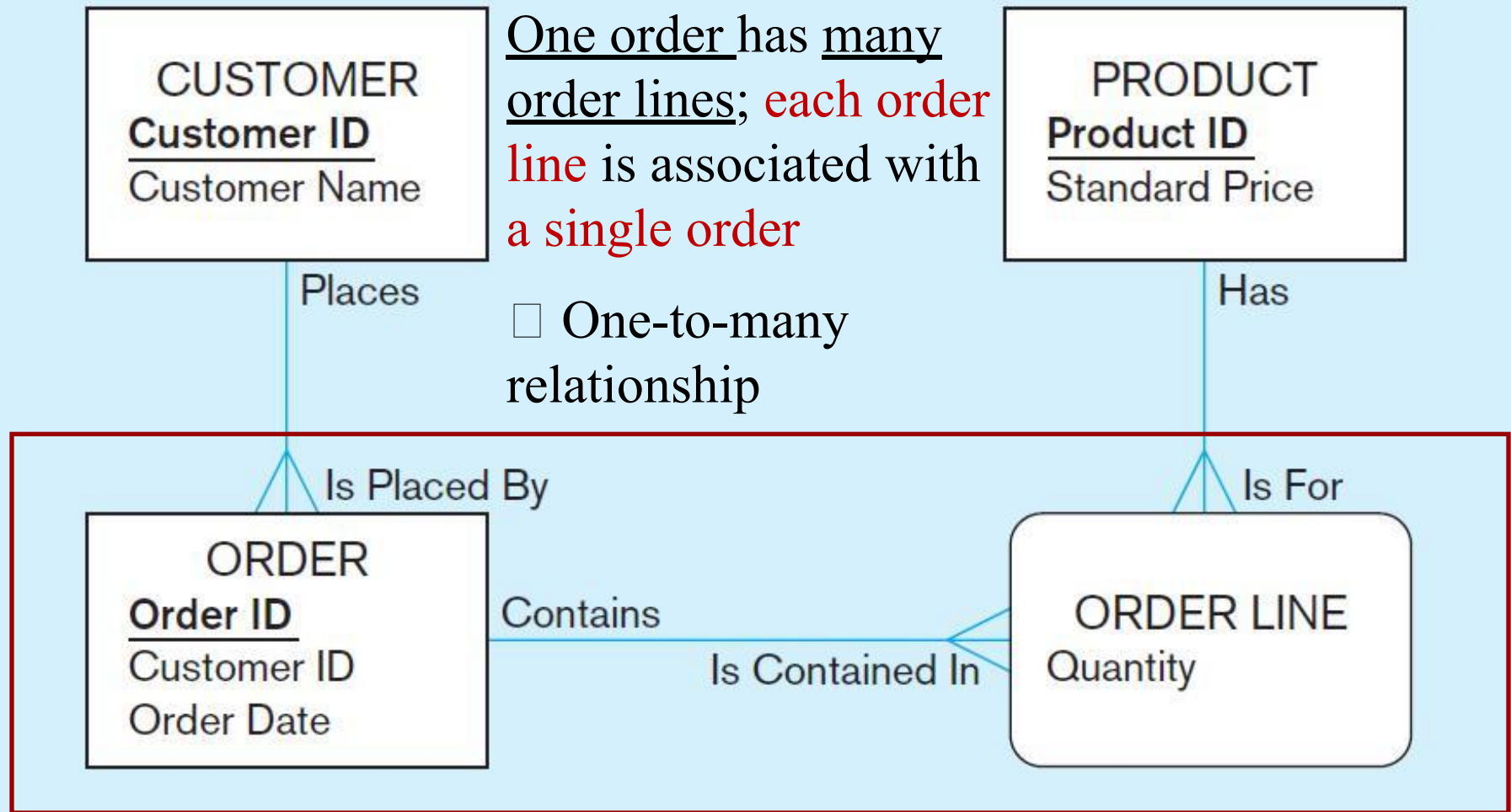
Note the expression of the relationship

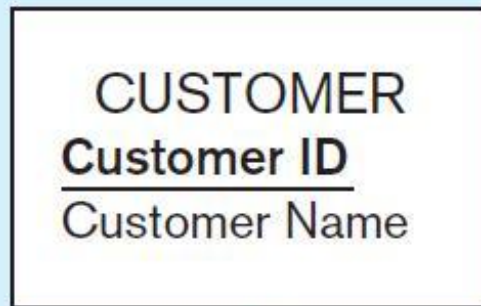


One customer
may place many
orders, but
each order is
placed by **a single**
customer

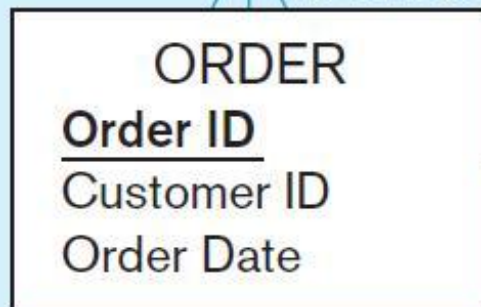
□ One-to-many
relationship

Note the expression of the relationship





Places



Is Placed By

Contains

Is Contained In



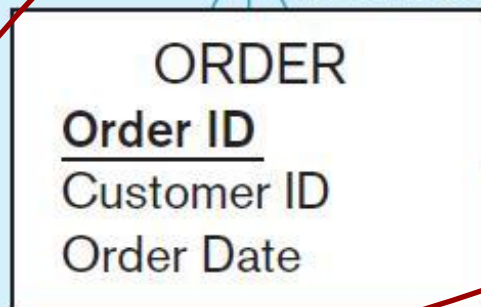
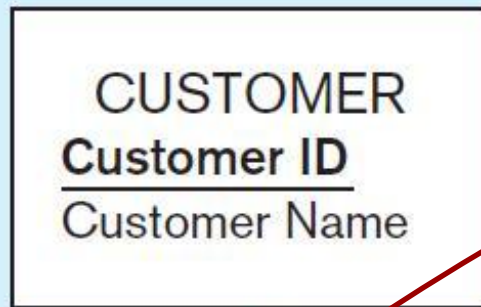
Has



Is For

One product can be in many order lines, **each order line** refers to **a single product**

□ One-to-many relationship



Places

Has

Is Placed By

Is For

Contains

Is Contained In

Therefore, one order involves many products and one product is involved in many orders

□ Many-to-many relationship

A many-to-many relationship can be “decomposed” to two 1-M relationships

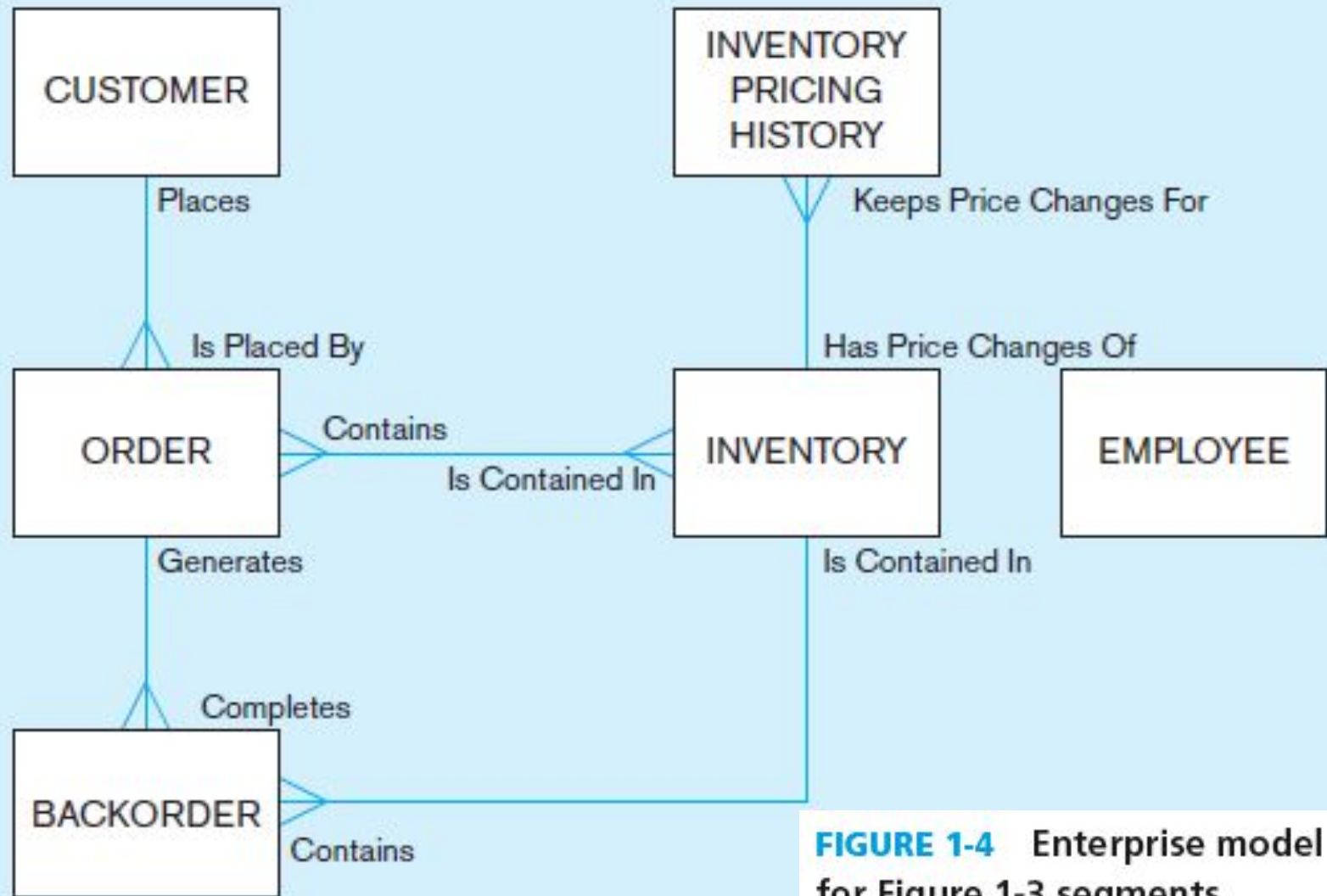
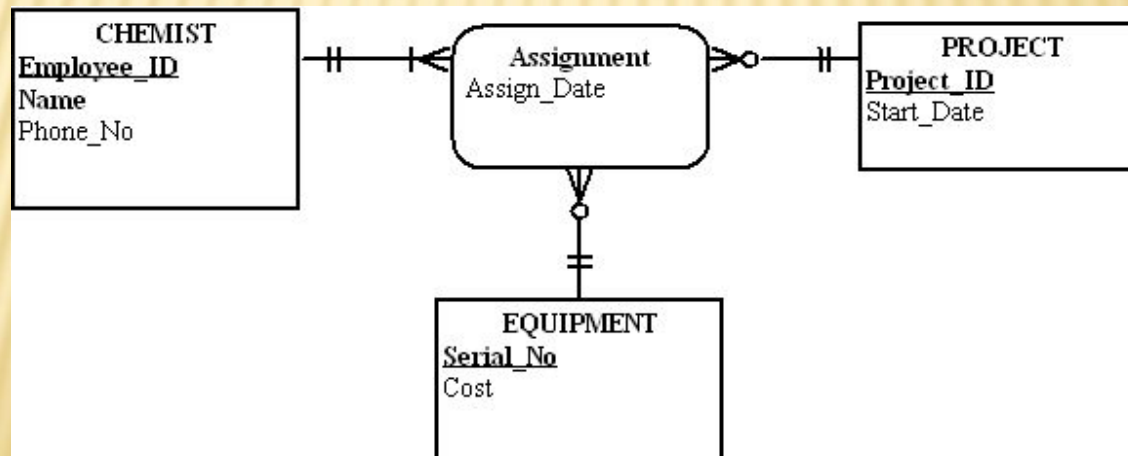
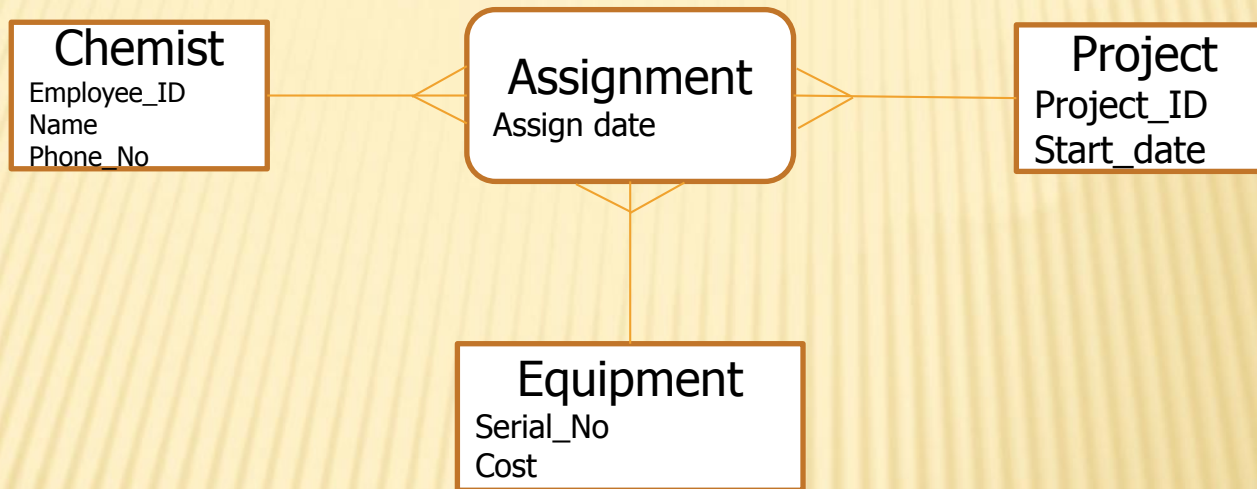


FIGURE 1-4 Enterprise model for Figure 1-3 segments

EXERCISE

A laboratory has several chemists who work on one or more projects. Chemists also may use certain kinds of equipment on each project. Attributes of CHEMIST include Employee_ID (identifier), Name, and Phone_No. Attributes of PROJECT include Project_ID (identifier) and Start_Date. Attributes of EQUIPMENT include Serial_No and Cost. The organization wishes to record Assign_Date—that is, the date when a given equipment item was assigned to a particular chemist working on a specified project. A chemist must be assigned to at least one project and one equipment item. A given equipment item need not be assigned, and a given project need not be assigned either a chemist or an equipment item. Provide good definitions for all of the relationships in this situation.

SOLUTION





ENTERPRISE DATA MODEL

- First step in the database development process
- Specifies scope and general content
- Overall picture of organizational data at **high level of abstraction**
 - **Entity-relationship diagram (ERD)**
 - **Descriptions of entity types**
 - **Relationships between entities**
 - **Business rules**

TWO APPROACHES TO DATABASE AND IS DEVELOPMENT

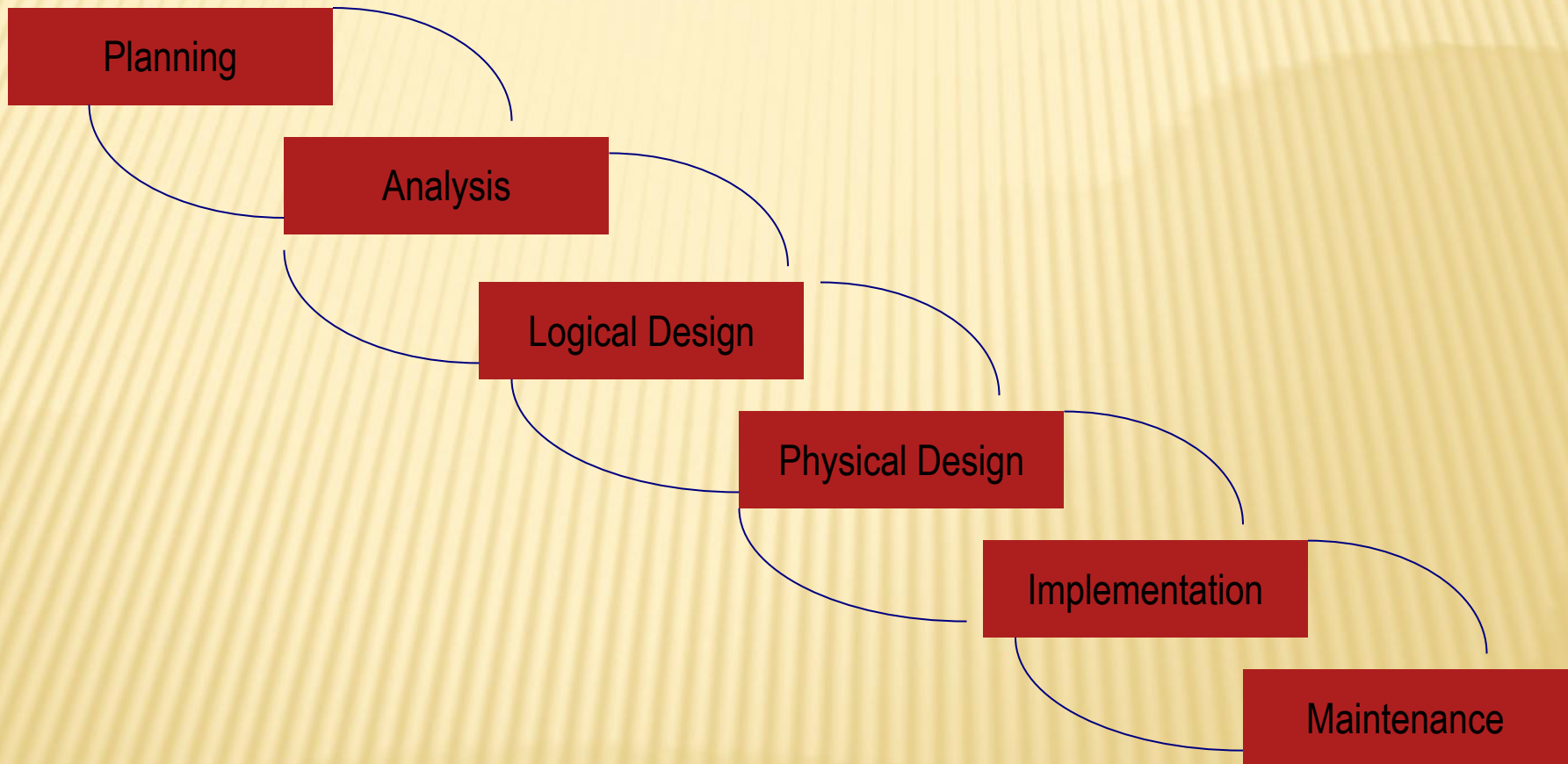
□ SDLC

- System Development Life Cycle
- Detailed, well-planned development process
- Time-consuming, but comprehensive
- Long development cycle

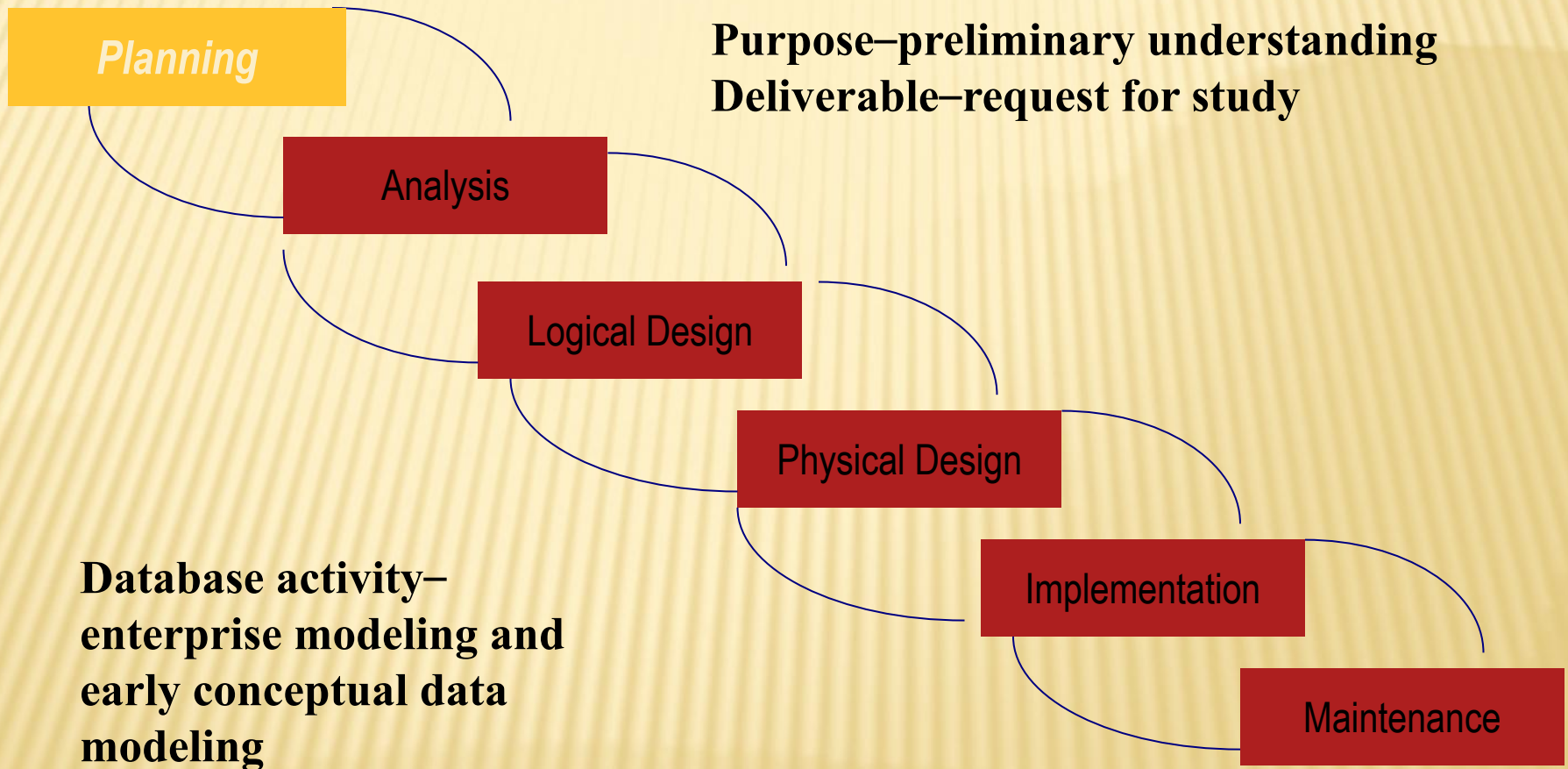
□ Prototyping

- Rapid application development (RAD)
- cursory attempt at conceptual data modeling
- Define database during development of initial prototype
- Repeat implementation and maintenance activities with new prototype versions

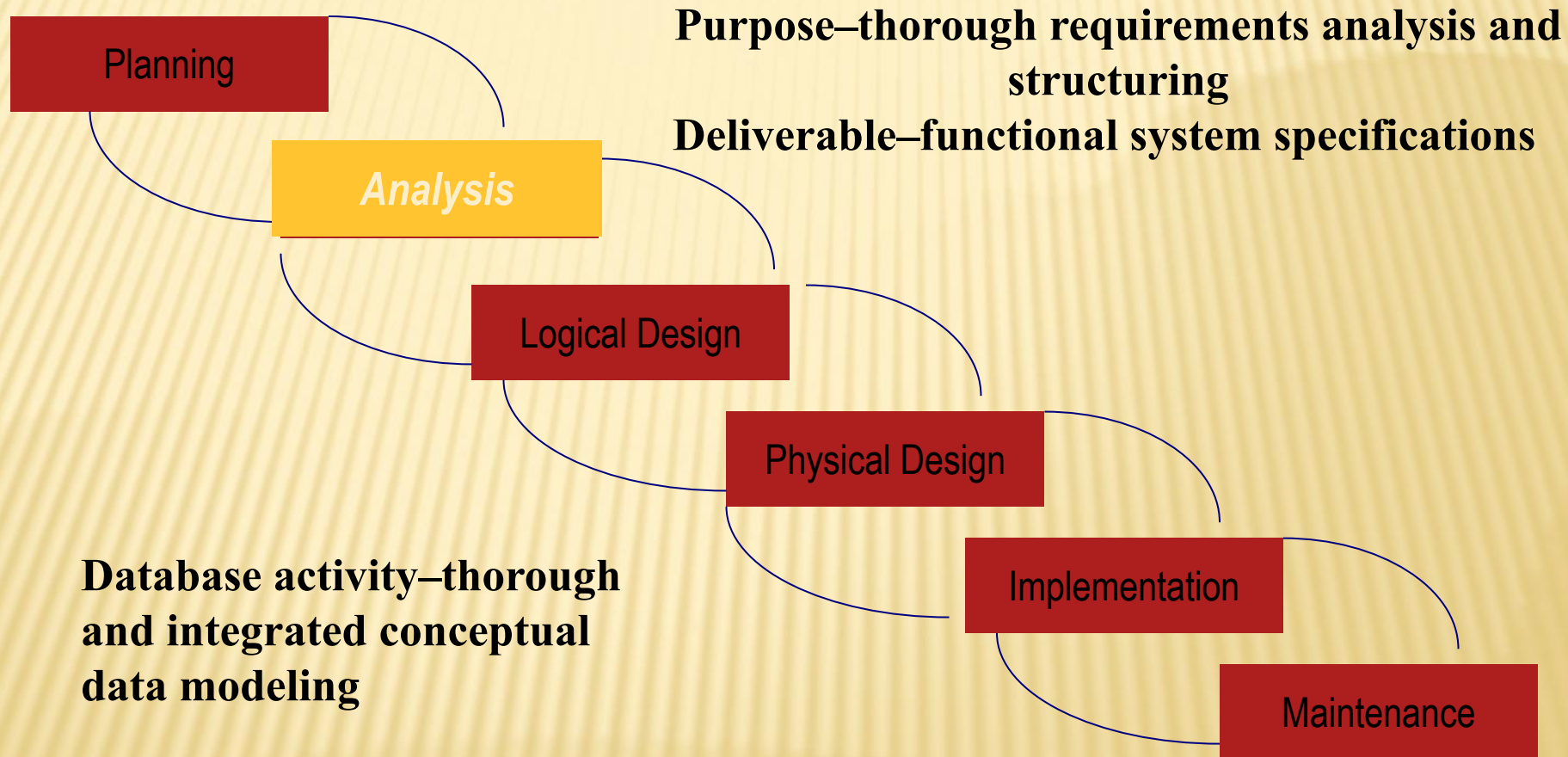
SYSTEMS DEVELOPMENT LIFE CYCLE (SEE ALSO FIGURE 1-7)



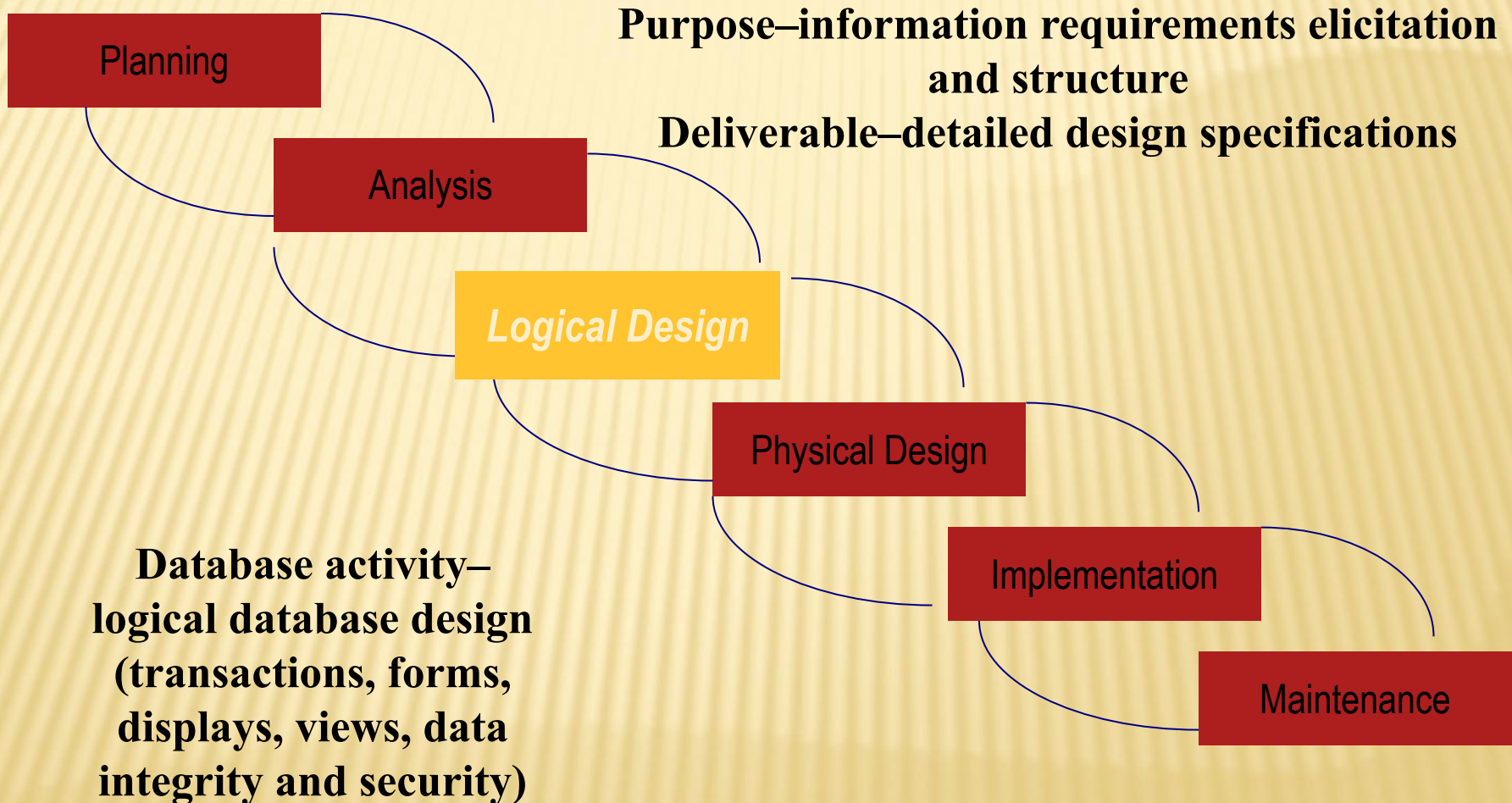
SYSTEMS DEVELOPMENT LIFE CYCLE (SEE ALSO FIGURE 1-7) (CONT.)



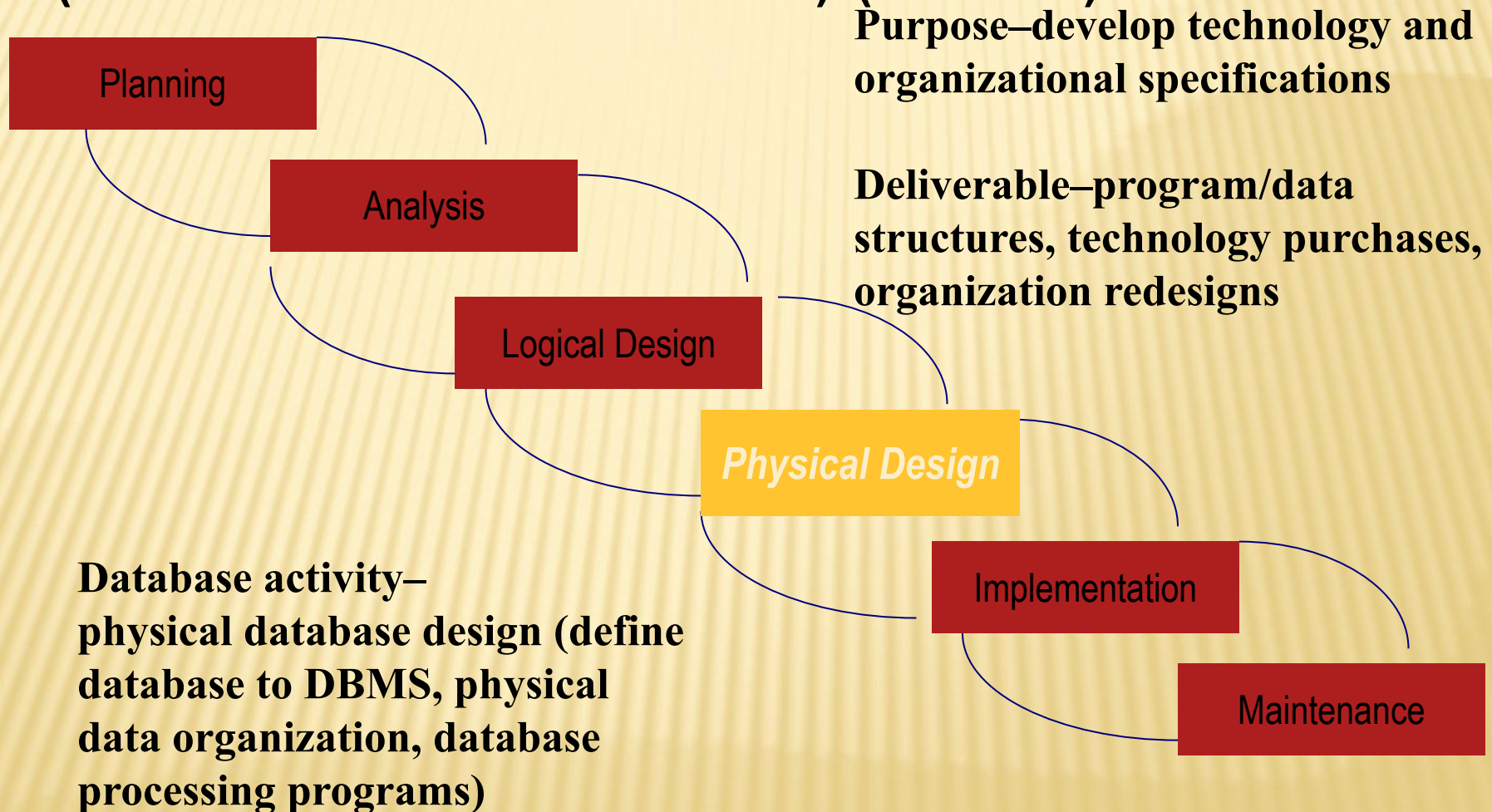
SYSTEMS DEVELOPMENT LIFE CYCLE (SEE ALSO FIGURE 1-7) (CONT.)



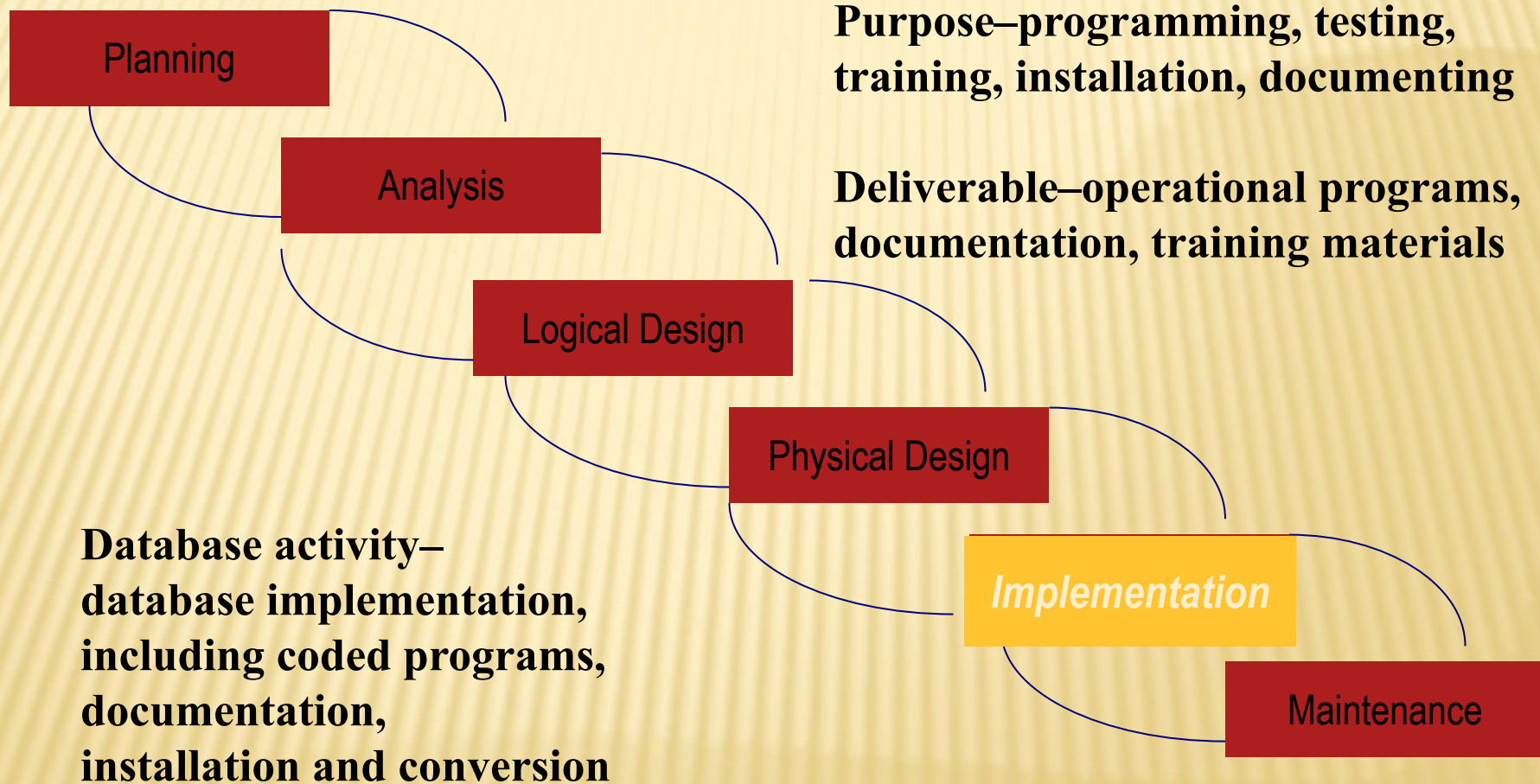
SYSTEMS DEVELOPMENT LIFE CYCLE (SEE ALSO FIGURE 1-7) (CONT.)



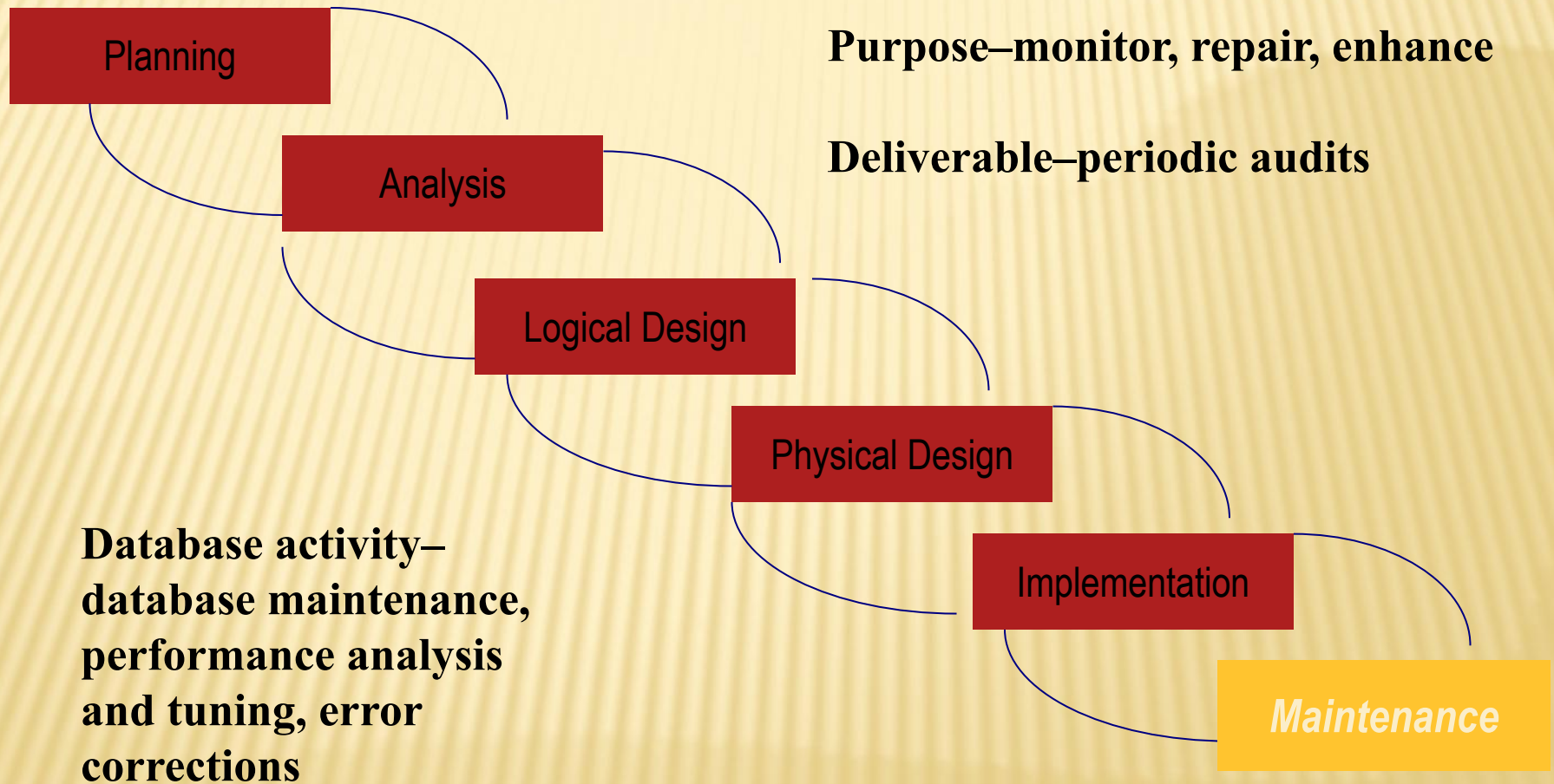
SYSTEMS DEVELOPMENT LIFE CYCLE (SEE ALSO FIGURE 1-7) (CONT.)



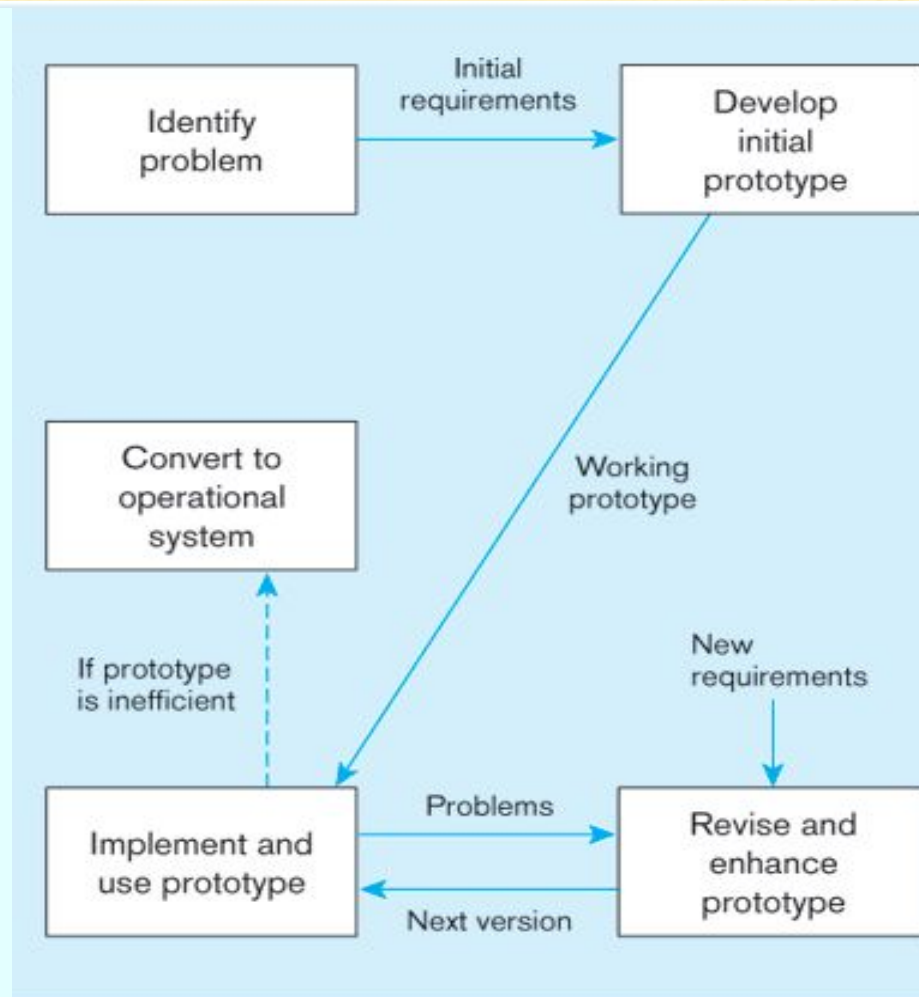
SYSTEMS DEVELOPMENT LIFE CYCLE (SEE ALSO FIGURE 1-7) (CONT.)



SYSTEMS DEVELOPMENT LIFE CYCLE (SEE ALSO FIGURE 1-7) (CONT.)



PROTOTYPING DATABASE METHODOLOGY (FIGURE 1-8)

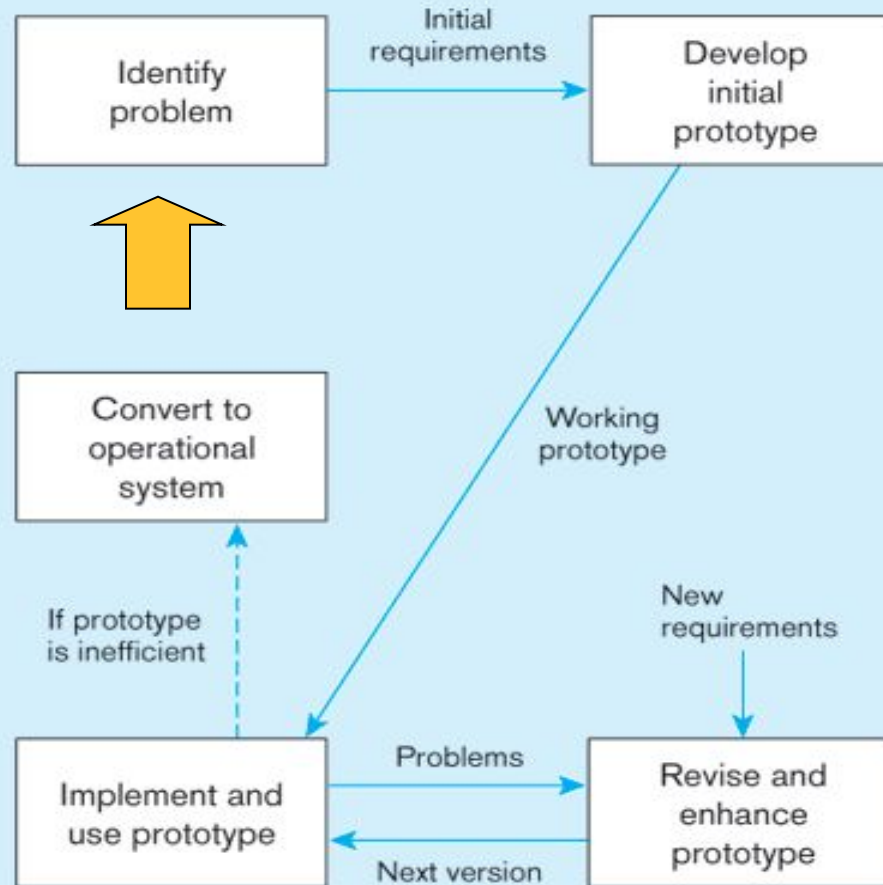


Prototyping is a classical Rapid Application Development (RAD) approach

PROTOTYPING DATABASE METHODOLOGY (FIGURE 1-8)

Conceptual data modeling

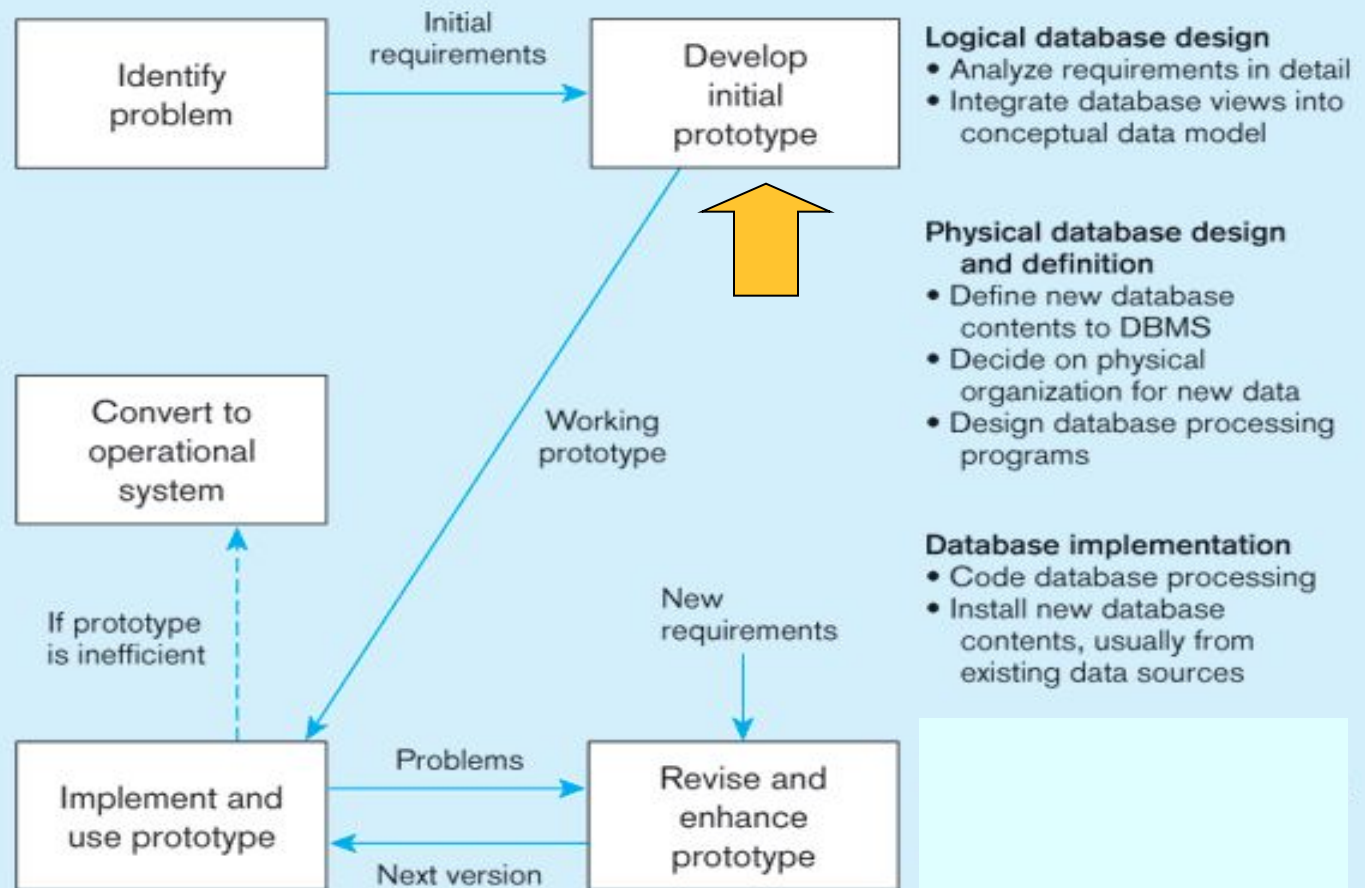
- Analyze requirements
- Develop preliminary data model



PROTOTYPING DATABASE METHODOLOGY (FIGURE 1-8)

Conceptual data modeling

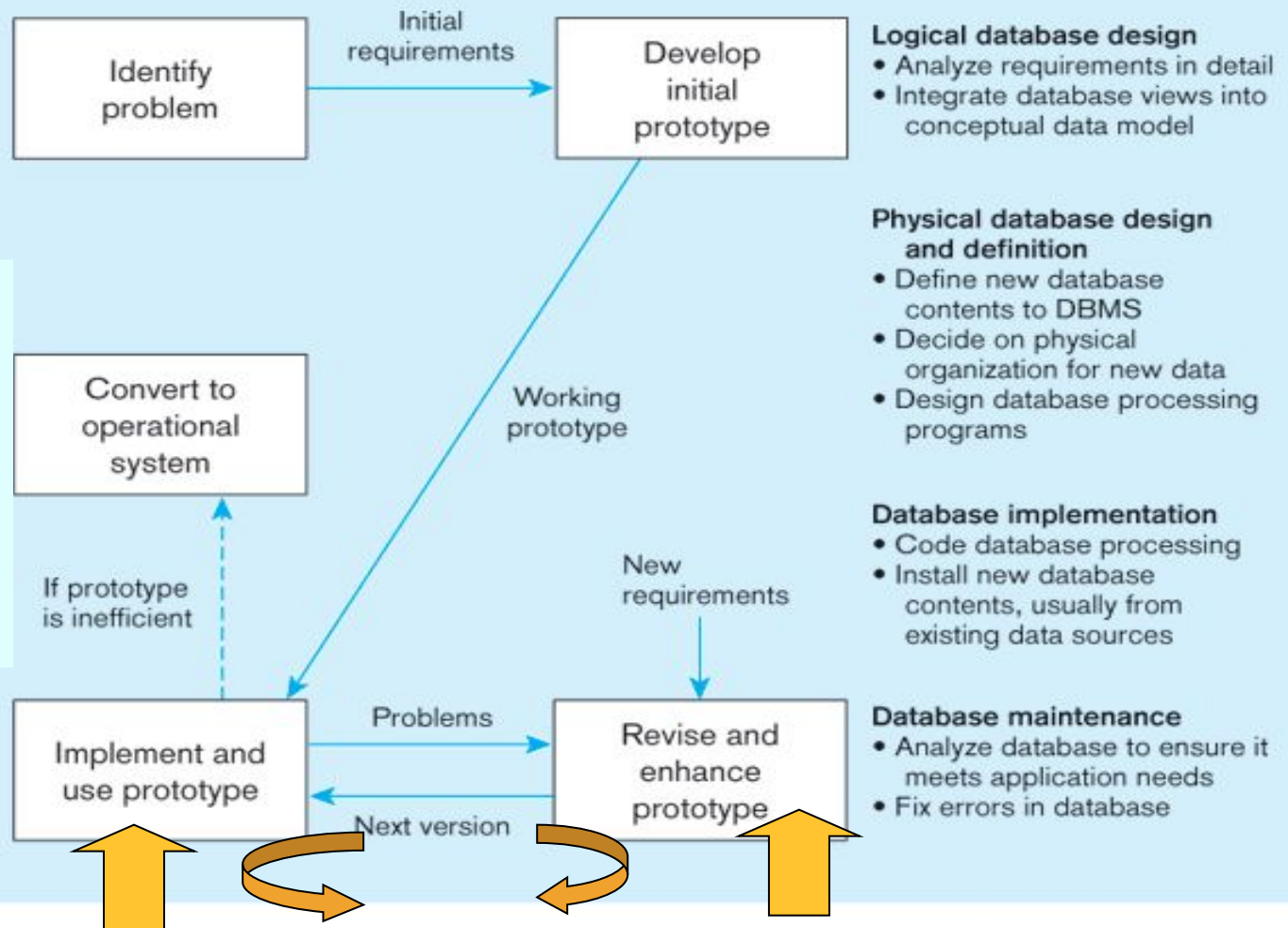
- Analyze requirements
- Develop preliminary data model



PROTOTYPING DATABASE METHODOLOGY (FIGURE 1-8)

Conceptual data modeling

- Analyze requirements
- Develop preliminary data model



PROTOTYPING DATABASE METHODOLOGY (FIGURE 1-8)

Conceptual data modeling

- Analyze requirements
- Develop preliminary data model

Database maintenance

- Tune database for improved performance
- Fix errors in database

Identify problem

Initial requirements

Develop initial prototype

Logical database design

- Analyze requirements in detail
- Integrate database views into conceptual data model

Physical database design and definition

- Define new database contents to DBMS
- Decide on physical organization for new data
- Design database processing programs

Database implementation

- Code database processing
- Install new database contents, usually from existing data sources

Database maintenance

- Analyze database to ensure it meets application needs
- Fix errors in database

Convert to operational system

If prototype is inefficient

Implement and use prototype

Problems

Revise and enhance prototype

Next version

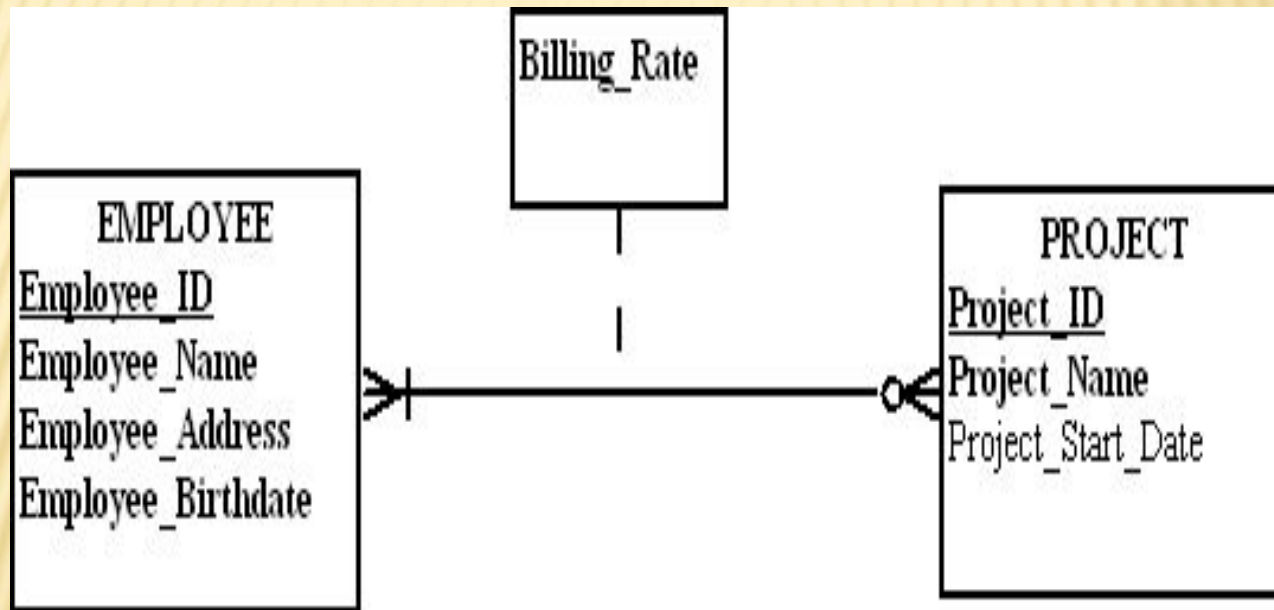
Working prototype

New requirements

EXERCISE

A company has a number of employees. The attributes of EMPLOYEE include Employee_ID (identifier), Name, Address, and Birthdate. The company also has several projects. Attributes of PROJECT include Project_ID (identifier), Project_Name, and Start_Date. Each employee may be assigned to one or more projects, or may not be assigned to a project. A project must have at least one employee assigned and may have any number of employees assigned. An employee's billing rate may vary by project, and the company wishes to record the applicable billing rate (Billing_Rate) for each employee when assigned to a particular project. Do the attribute names in this description follow the guidelines for naming attributes? If not, suggest better names.

EXERCISE



GENERAL STEPS TO CREATE AN ERD

- **Identify the entity**
- **Identify the entity's attributes**
- **Identify the Primary Keys**
- **Identify the relation between entities**
- **Identify the Cardinality constraint**
- **Draw the ERD**
- **Check the ERD**

DEVELOPING AN ERD

The process has ten steps:

1. Identify Entities
2. Find Relationships
3. Draw Rough ERD
4. Fill in Cardinality
5. Define Primary Keys
6. Draw Key-Based ERD
7. Identify Attributes
8. Map Attributes
9. Draw fully attributed ERD
10. Check Results

A SIMPLE EXAMPLE

A company has several departments. Each department has a supervisor and at least one employee. Employees must be assigned to at least one, but possibly more departments. At least one employee is assigned to a project, but an employee may be on vacation and not assigned to any projects. The important data fields are the names of the departments, projects, supervisors and employees, as well as the supervisor and employee number and a unique project number.

IDENTIFY ENTITIES

- One approach to this is to work through the information and highlight those words which you think correspond to entities.
- **company** has several **departments**. Each department has a **supervisor** and at least one **employee**. Employees must be assigned to at least one, but possibly more departments. At least one employee is assigned to a **project**, but an employee may be on vacation and not assigned to any projects. The important data fields are the names of the departments, projects, supervisors and employees, as well as the supervisor and employee number and a unique project number.
- A true entity should have more than one instance

FIND RELATIONSHIPS

- Aim is to identify the associations, the connections between pairs of entities.
- A simple approach to do this is using a relationship matrix (table) that has rows and columns for each of the identified entities.

	Department	Employee	Supervisor	Project
Department				
Employee				
Supervisor				
Project				

FIND RELATIONSHIPS (CONTD.)

- Go through each cell and decide whether or not there is an association. For example, the first cell on the second row is used to indicate if there is a relationship between the entity "Employee" and the entity "Department".

	Department	Employee	Supervisor	Project
Department		Is assigned	Run by	
Employee	Belongs to			Works on
Supervisor	Runs			
Project		Uses		

IDENTIFIED RELATIONSHIPS

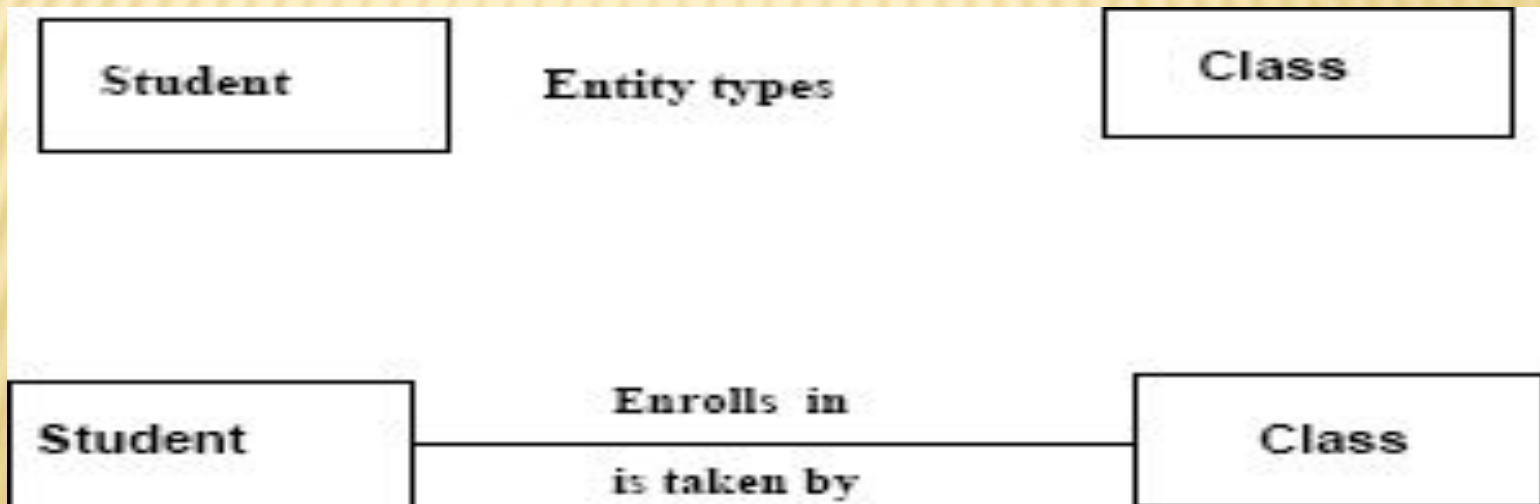
Names placed in the cells are meant to capture/describe the relationships. So you can use them like this

- A Department is **assigned** an employee
- A Department is **run** by a supervisor
- An employee **belongs** to a department
- An employee **works** on a project
- A supervisor **runs** a department
- A project **uses** an employee

DRAW ROUGH ERD

Draw a diagram and:

- Place all the **entities** in **rectangles**
- Use **diamonds** and **lines** to represent the **relationships** between **entities**.
- General Examples



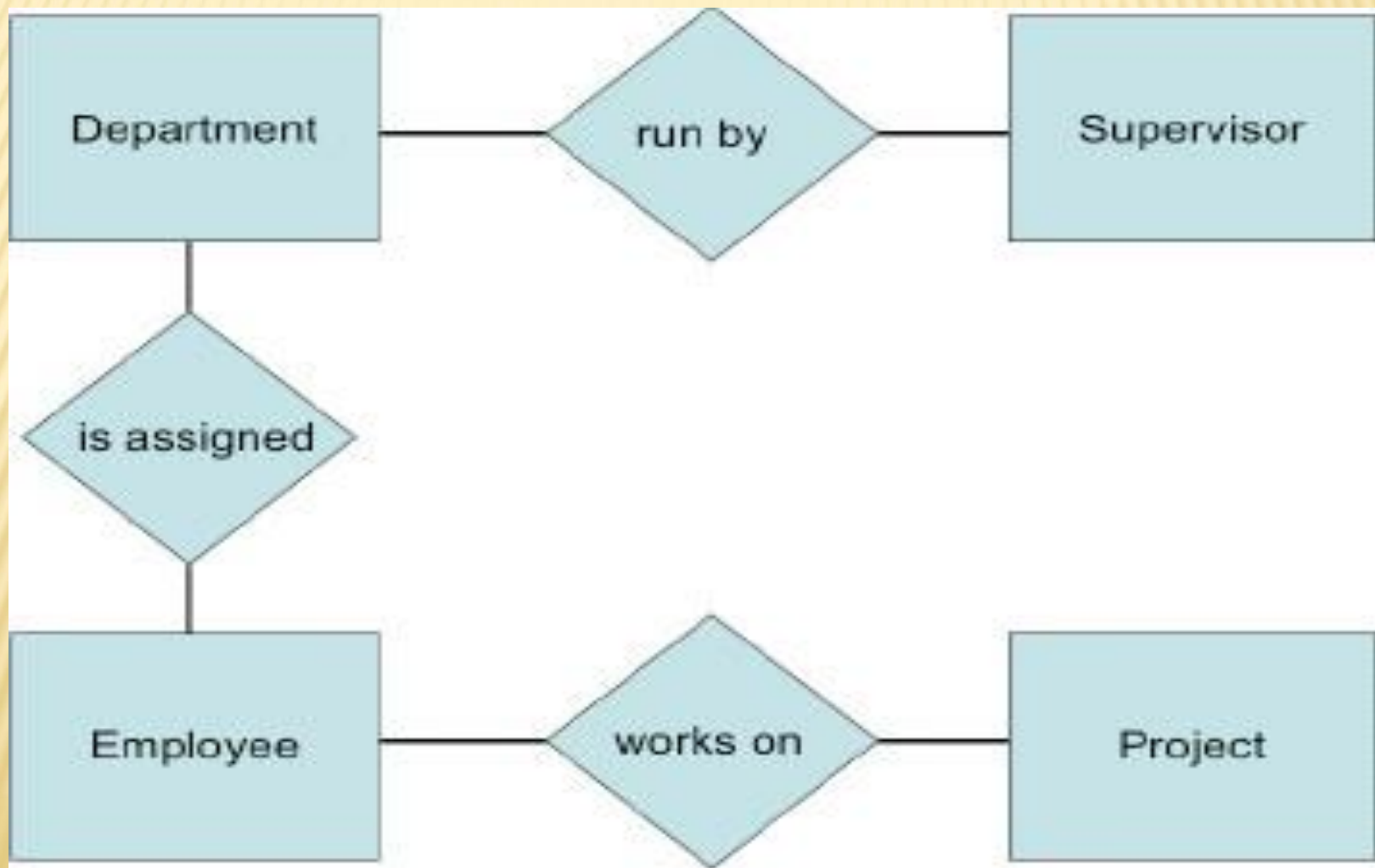
DRAWING ROUGH ERD (CONTD.)



DRAWING ROUGH ERD (CONTD.)



DRAWING ROUGH ERD (CONTD.)



FILL IN CARDINALITY

- Supervisor
 - Each department has **one** supervisor.
- Department
 - Each supervisor has **one** department.
 - Each employee can belong to **one or more** departments
- Employee
 - Each department must have **one or more** employees
 - Each project must have **one or more** employees
- Project
 - Each employee can have **0 or more** projects.

FILL IN CARDINALITY (CONTD.)

The cardinality of a relationship can only have the following values

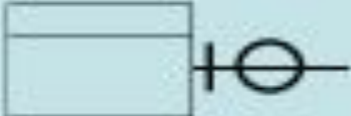
- One and only one

- One or more

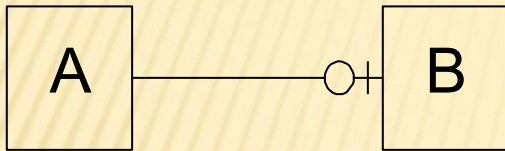
- Zero or more

- Zero or one

CARDINALITY NOTATION

Symbol	Meaning
 The symbol consists of a light blue rectangle with a horizontal line across its top. To the right of the rectangle is a vertical line with two short horizontal bars (double bars) at its end, representing a mandatory one-to-one relationship.	One and only one
 The symbol consists of a light blue rectangle with a horizontal line across its top. To the right of the rectangle is a vertical line with a crow's foot notation (a line branching into three) at its end, representing a mandatory one-to-many relationship.	One or more
 The symbol consists of a light blue rectangle with a horizontal line across its top. To the right of the rectangle is a vertical line with a crow's foot notation (a line branching into three) at its end, followed by a small circle, representing an optional one-to-many relationship.	Zero or more
 The symbol consists of a light blue rectangle with a horizontal line across its top. To the right of the rectangle is a vertical line with a small circle at its end, representing an optional one-to-one relationship.	Zero or one

CARDINALITY EXAMPLES



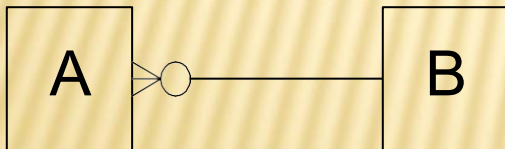
Each instance of A is related to a minimum of zero and a maximum of one instance of B



Each instance of B is related to a minimum of one and a maximum of one instance of A

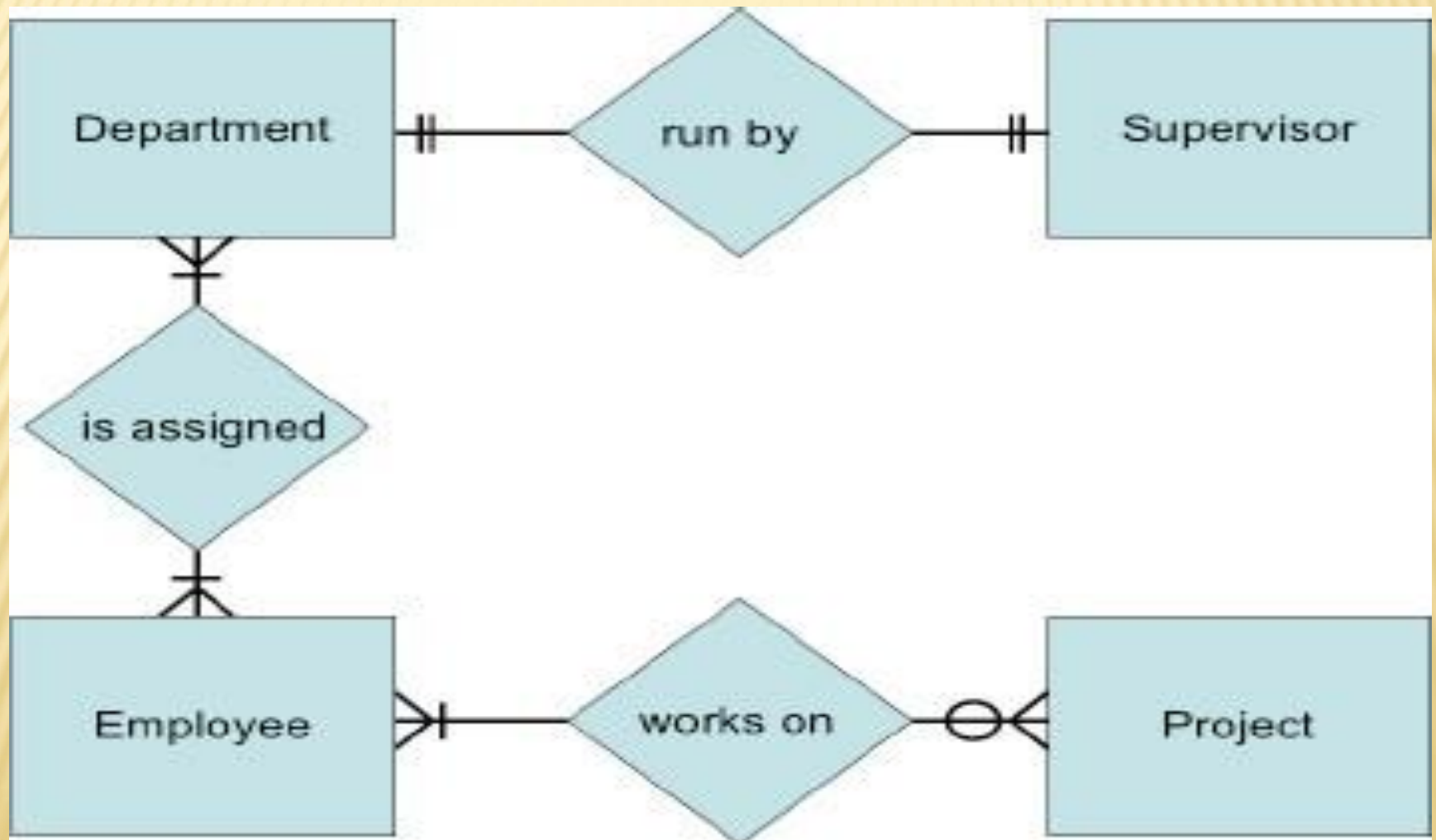


Each instance of A is related to a minimum of one and a maximum of many instances of B



Each instance of B is related to a minimum of zero and a maximum of many instances of A

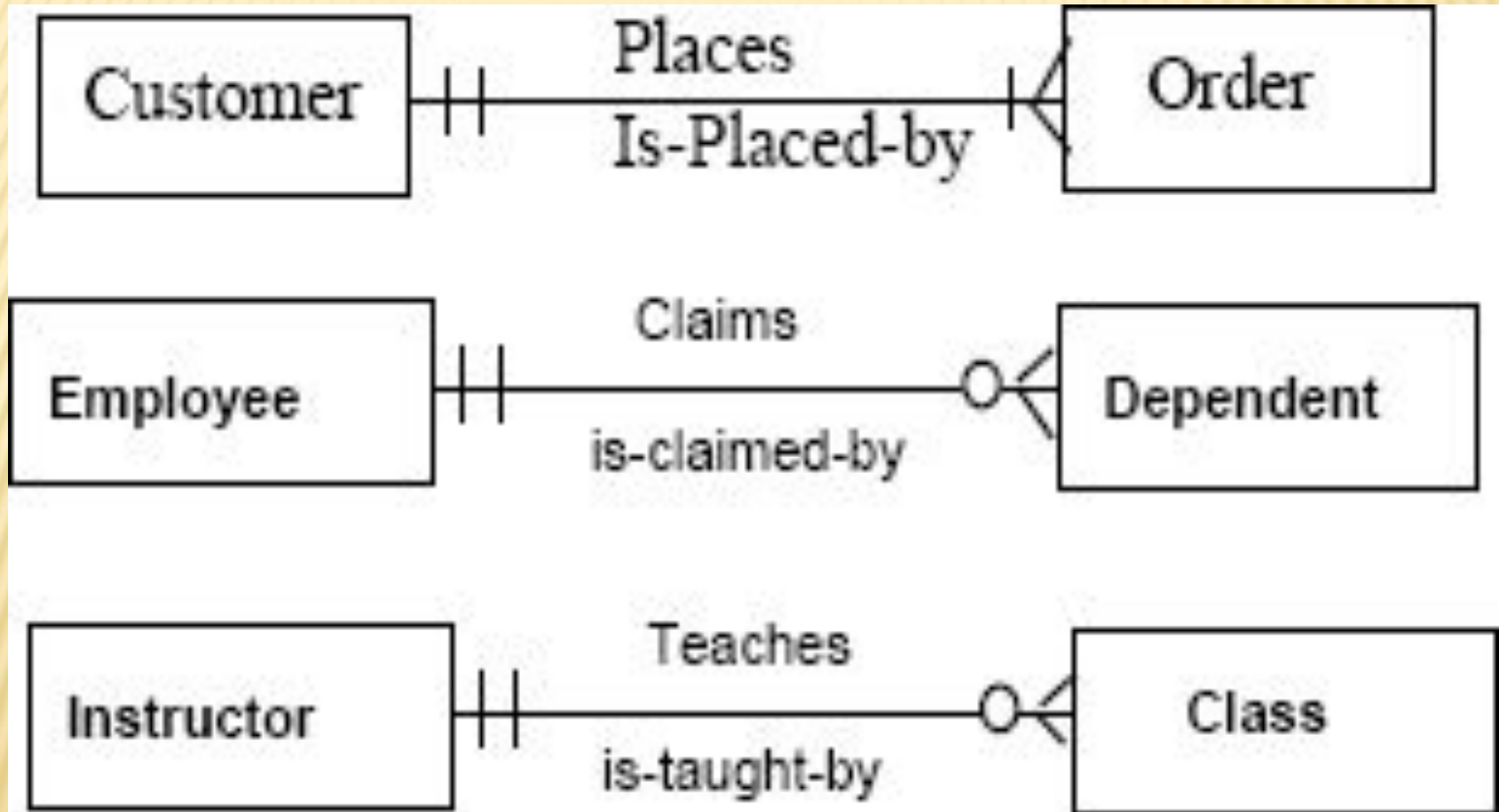
ERD WITH CARDINALITY



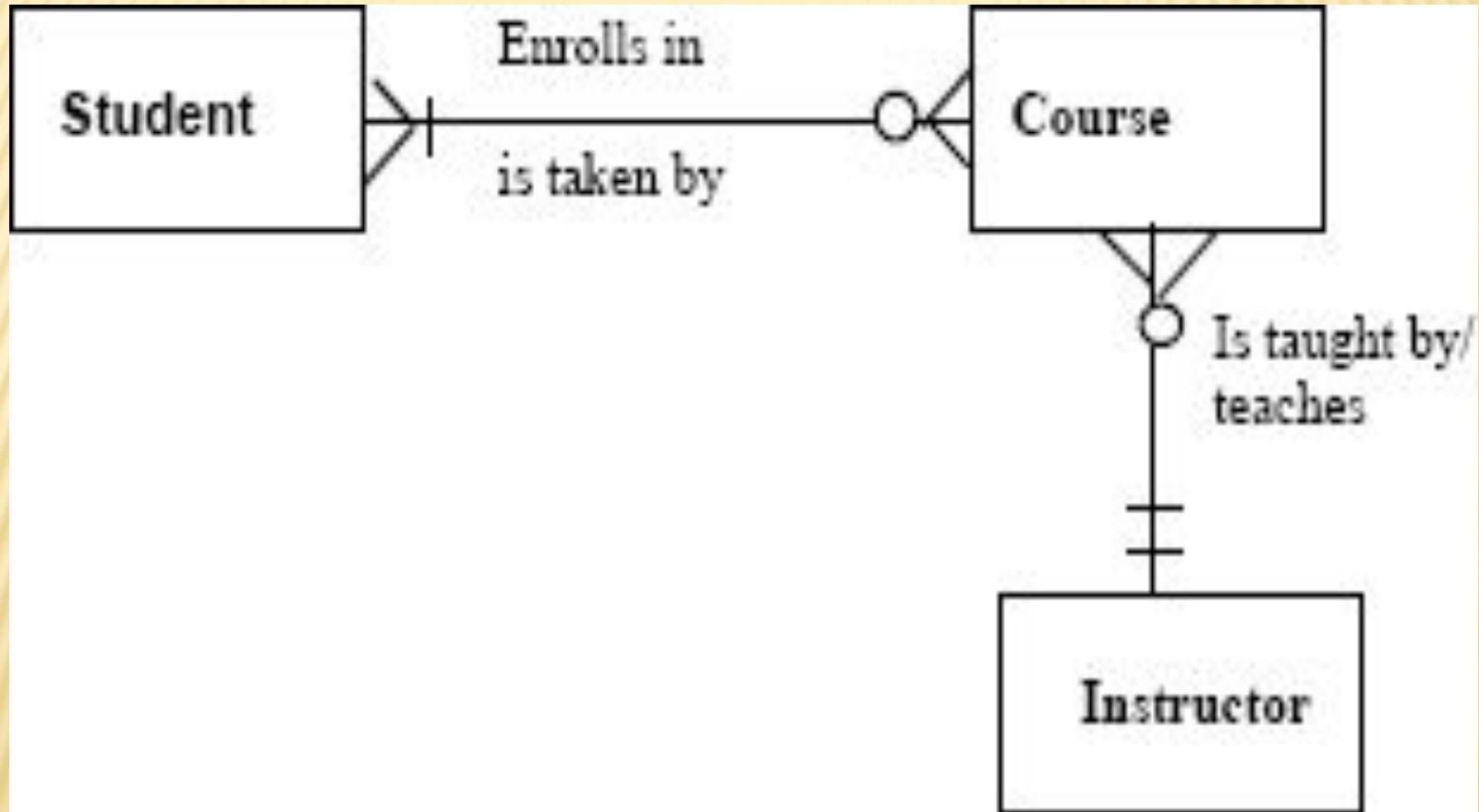
EXAMPLES

1. Order must be placed by a customer but a customer may places many orders.
2. Employee may claim to many dependents but dependent must claimed by an employee.
3. Instructor may teach to many classes but class must be taught by an instructor

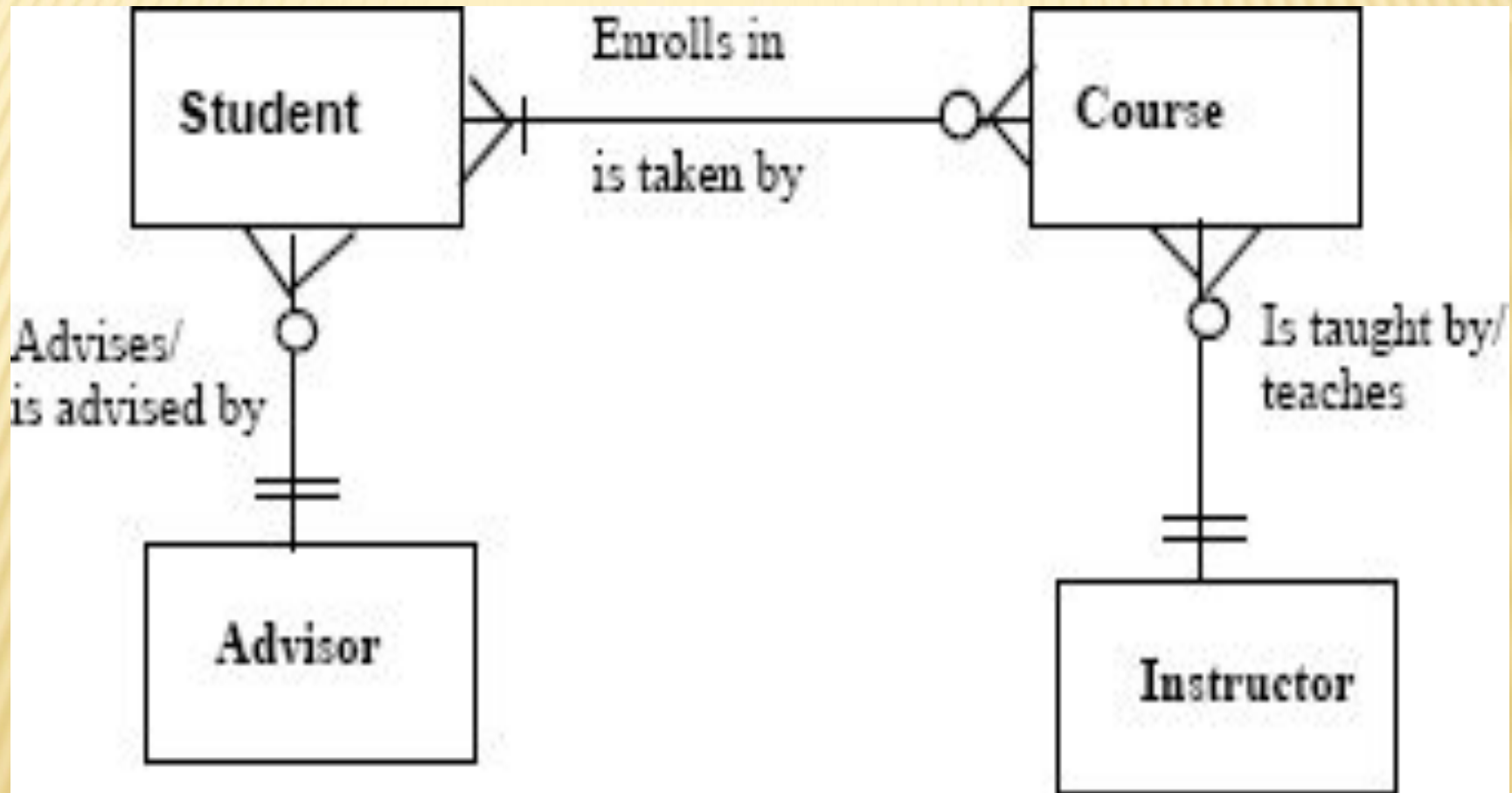
EXAMPLES



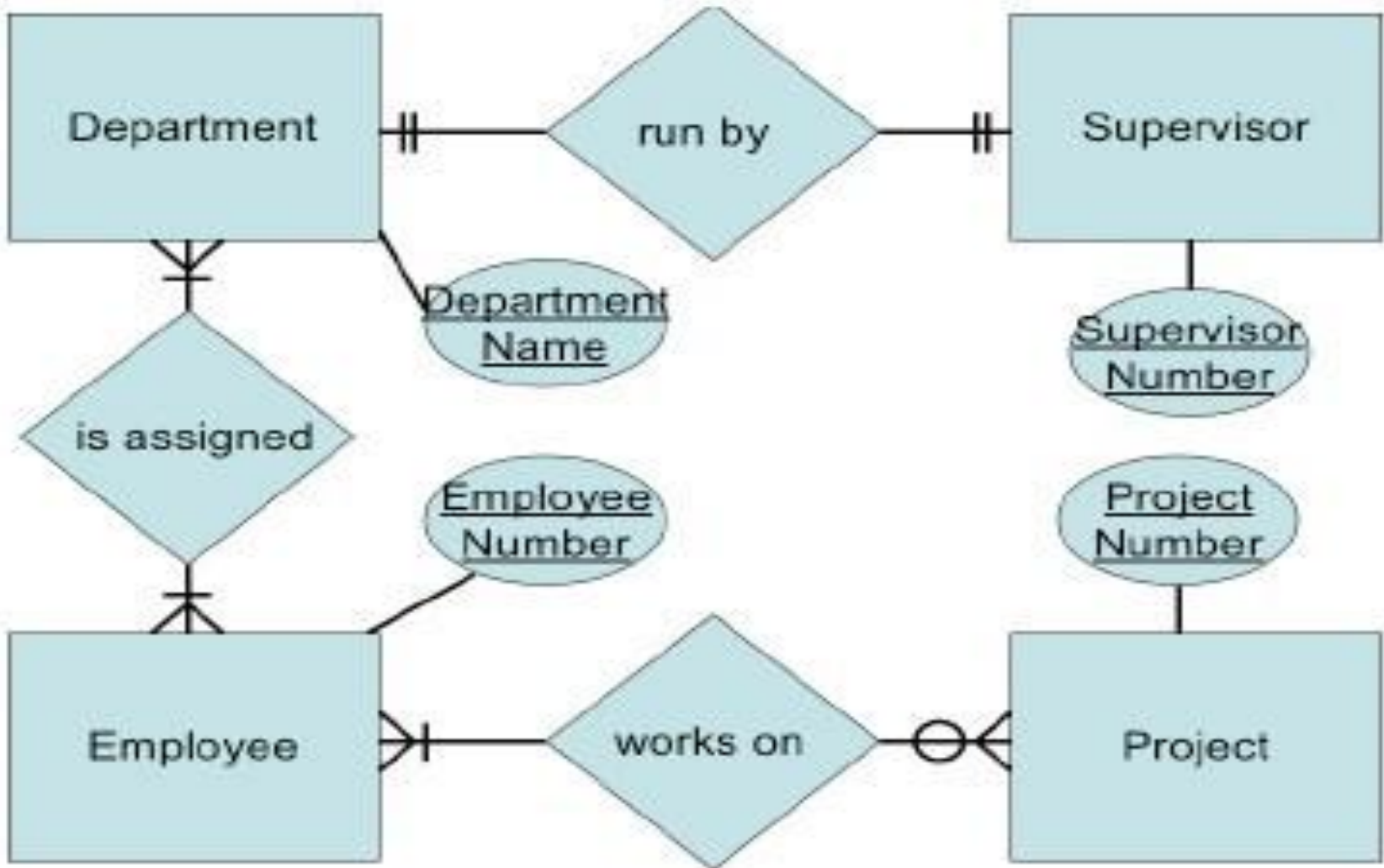
ERD FOR COURSE ENROLLMENT



ERD FOR COURSE REGISTRATION



ROUGH ERD PLUS PRIMARY KEYS



IDENTIFY ATTRIBUTES

- In this step we try to identify and name all the attributes essential to the system we are studying without trying to match them to particular entities.
- The best way to do this is to study the forms, files and reports currently kept by the users of the system and circle each data item on the paper copy.
- Cross out those which will not be transferred to the new system, extraneous items such as signatures, and constant information which is the same for all instances of the form (e.g. your company name and address). The remaining circled items should represent the attributes you need. You should always verify these with your system users. (Sometimes forms or reports are out of date.)
- The only attributes indicated are the names of the departments, projects, supervisors and employees, as well as the supervisor and employee NUMBER and a unique project number.

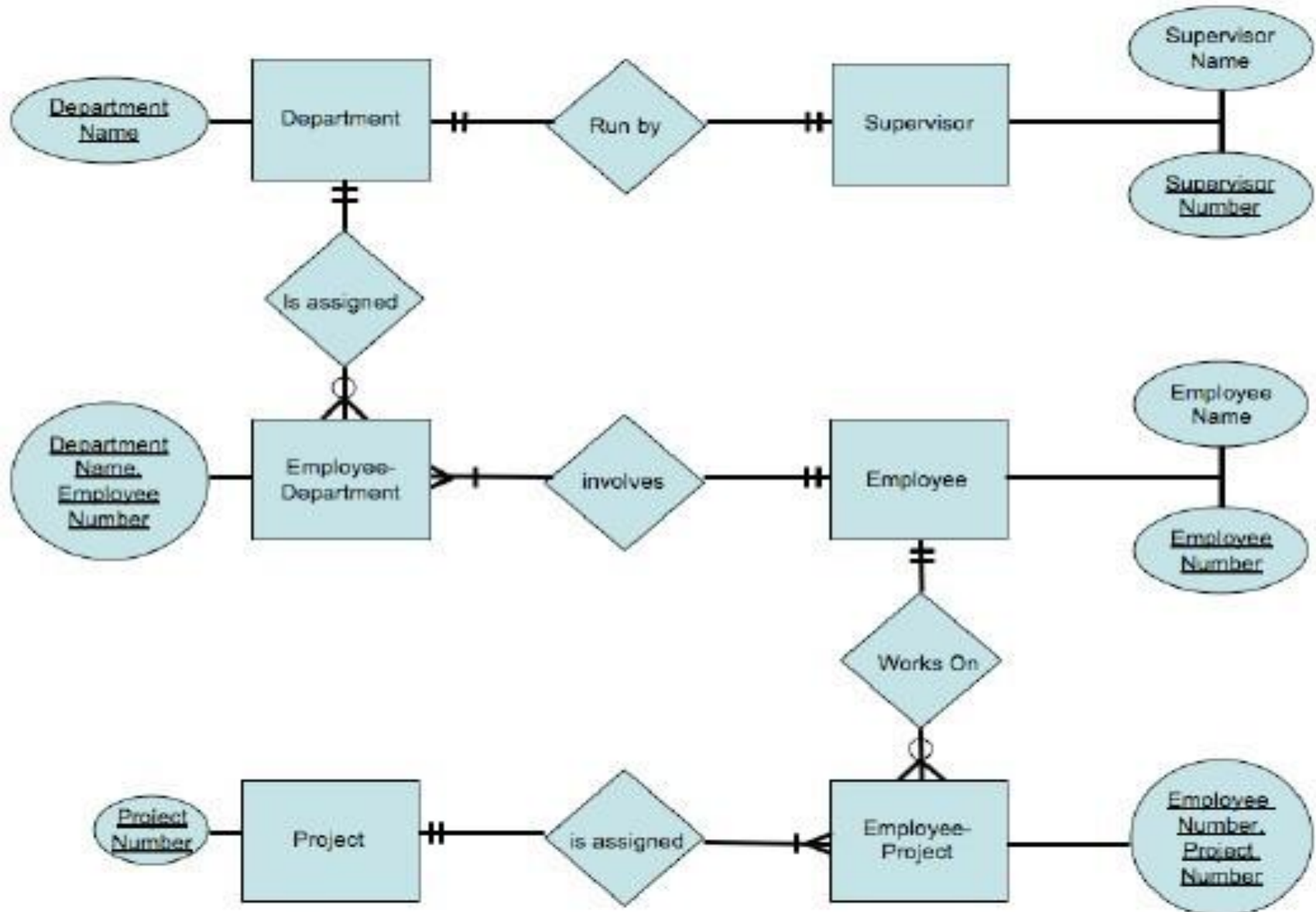
MAP ATTRIBUTES

- For each attribute we need to match it with exactly one entity. Often it seems like an attribute should go with more than one entity (e.g. Name). In this case you need to add a modifier to the attribute name to make it unique (e.g. Customer Name, Employee Name, etc.) or determine which entity an attribute "best" describes.
- If you have attributes left over without corresponding entities, you may have missed an entity and its corresponding relationships. Identify these missed entities and add them to the relationship matrix now.

MAP ATTRIBUTES (CONTD.)

Attribute	Entity	Attribute	Entity
Department Name	Department	Supervisor Number	Supervisor
Employee Number	Employee	Supervisor Name	Supervisor
Employee Name	Employee	Project Name	Project
		Project Number	Project

DRAW FULLY ATTRIBUTED ERD



CHECK ERD RESULTS

- Look at your diagram from the point of view of a system owner or user. **Is everything clear?**
- Check through the **Cardinality pairs**.
- Also, look over the list of attributes associated with each entity to see if anything has been omitted.